



A high-level introduction to Language Models

Sacha Davis
Feb 02 2026



The day the world was forever changed!

New chat

History is temporarily unavailable.
We're working to restore this feature as soon as possible.

Upgrade to Plus

Dark mode

Updates & FAQ

NEW

ChatGPT



Examples

"Explain quantum computing in simple terms" →

"Got any creative ideas for a 10 year old's birthday?" →

"How do I make an HTTP request in Javascript?" →



Capabilities

Remembers what user said earlier in the conversation

Allows user to provide follow-up corrections

Trained to decline inappropriate requests



Limitations

May occasionally generate incorrect information

May occasionally produce harmful instructions or biased content

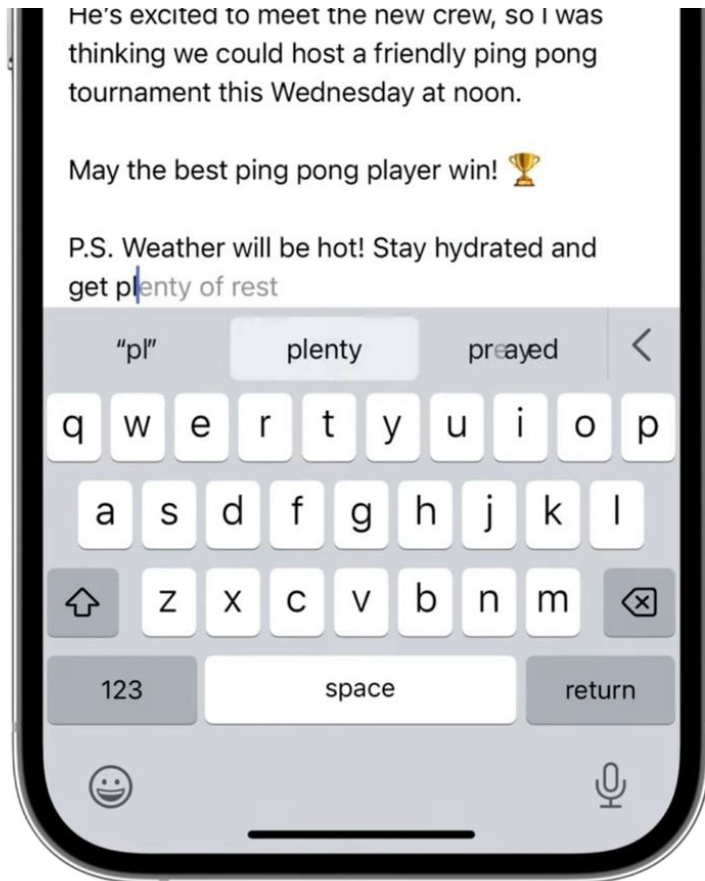
Limited knowledge of world and events after 2021

2

**So ... what is an LLM and how do
you make one?**

Long before Language Models, there were intelligent keyboards!

Helping the user by
predicting the next word
on a mobile keyboard.



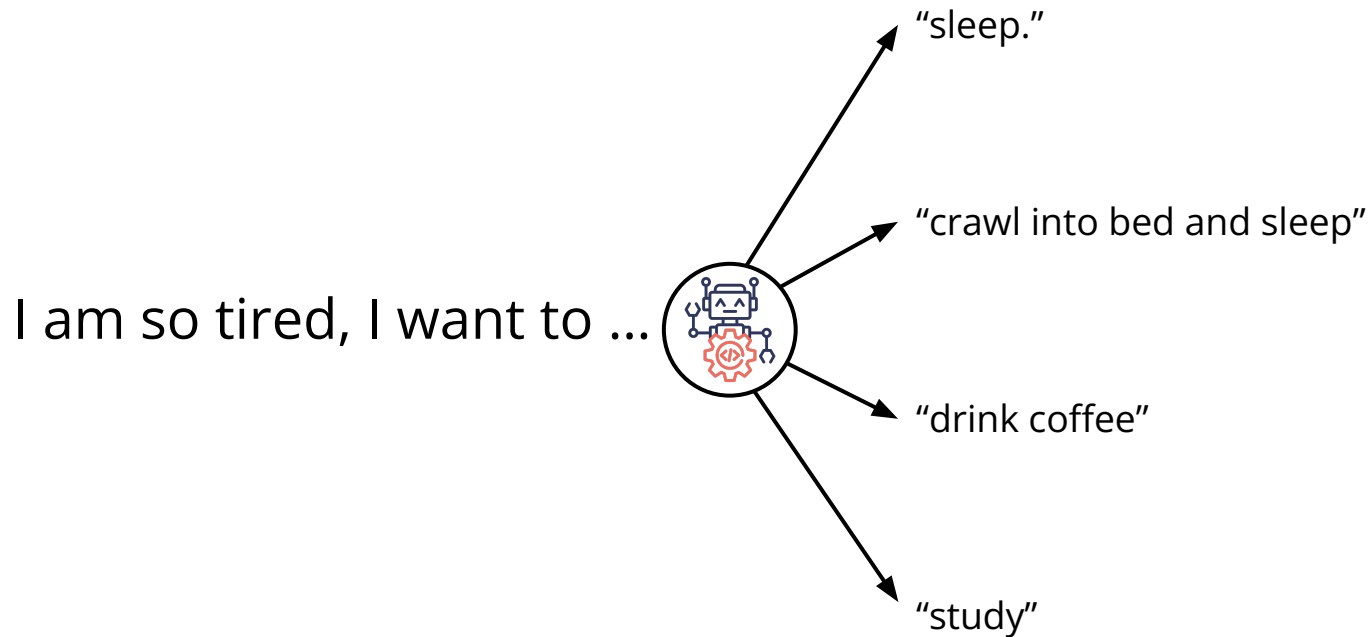
Step 1

Next Word Prediction

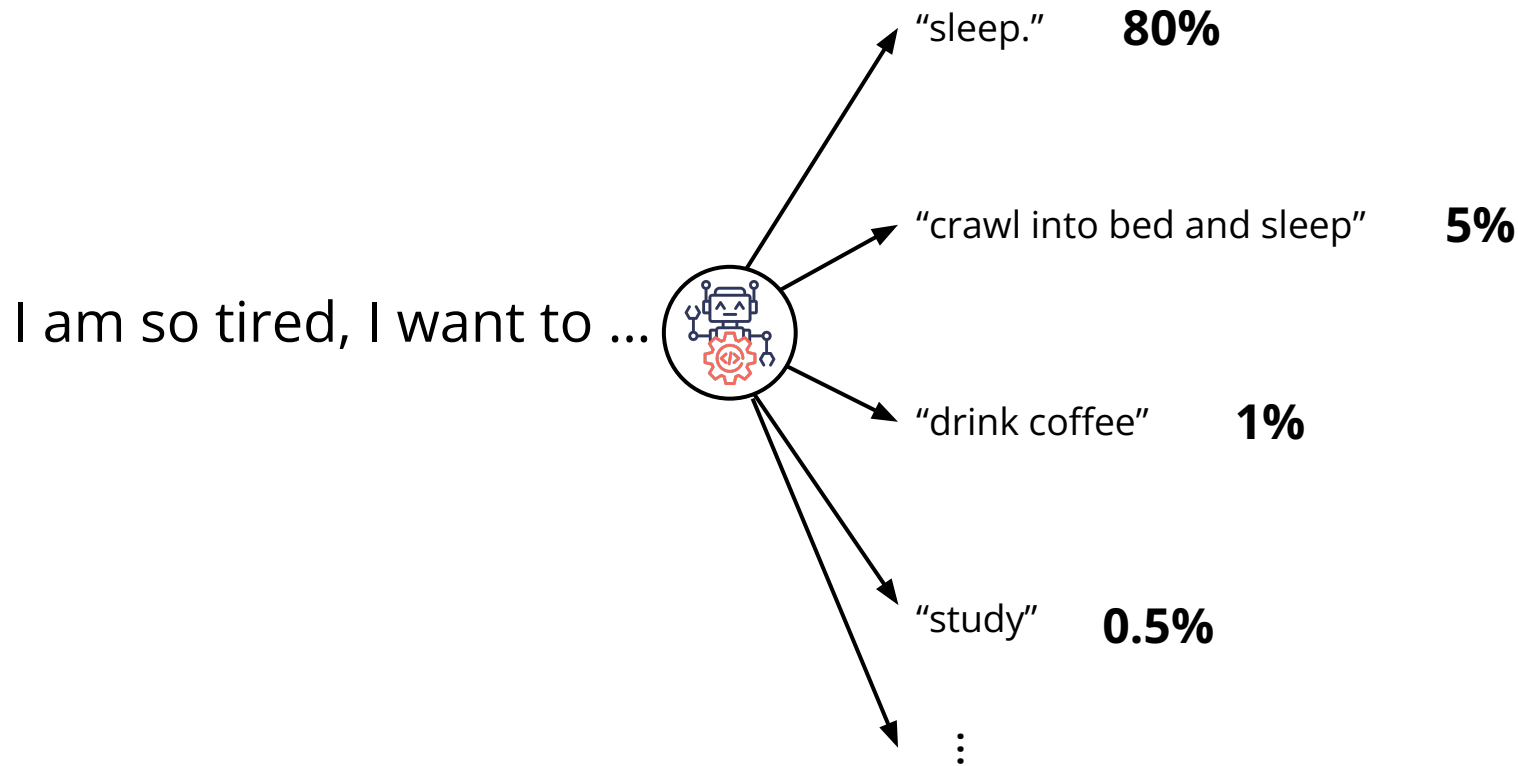
Next word prediction problem: What should come next after seeing a given sequence of words?

I am so tired, I want to ...

Prediction the next **word** correctly requires understanding the **world**!

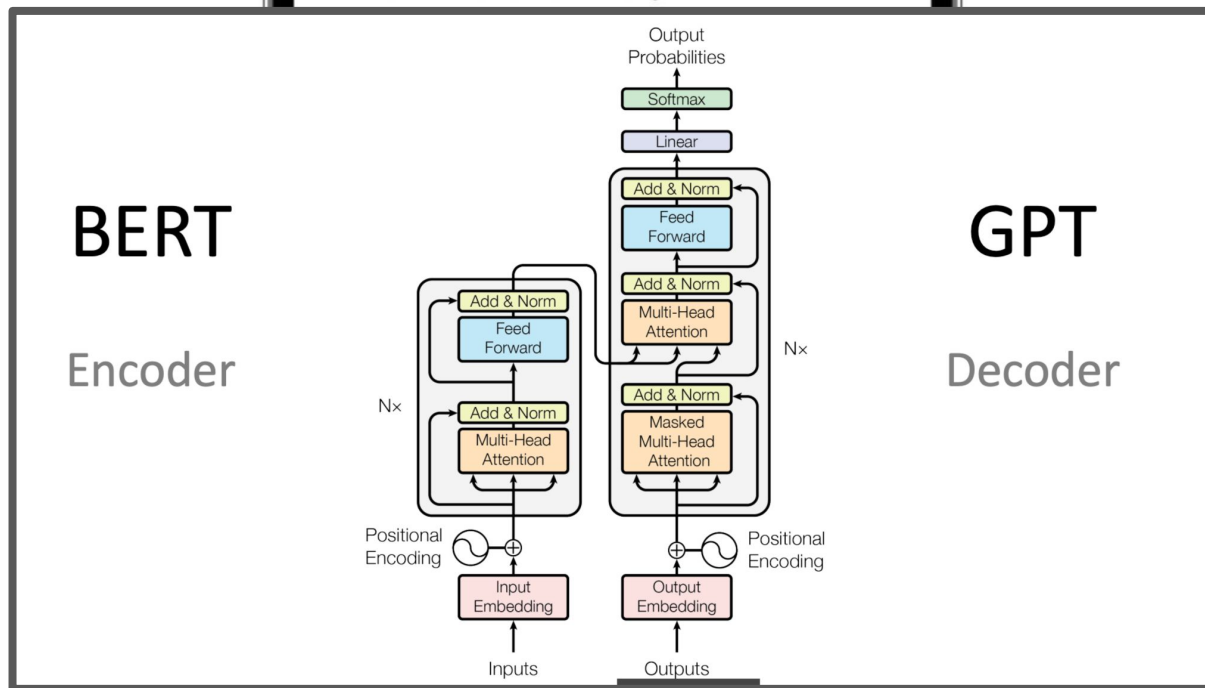


Correct prediction of the next word requires understanding the **world**!

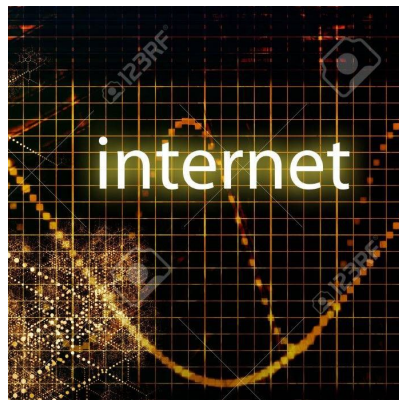


An LLM is a next-word predictor, scaled up with Transformers and vast amounts of data!

He's excited to meet the new crew, so I was thinking we could host a friendly ping pong tournament this Wednesday at noon.



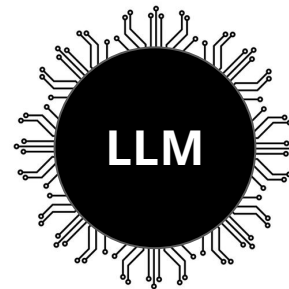
An LLM is a next-word predictor, scaled up with Transformers and vast amounts of data!



Copy of the
Internet



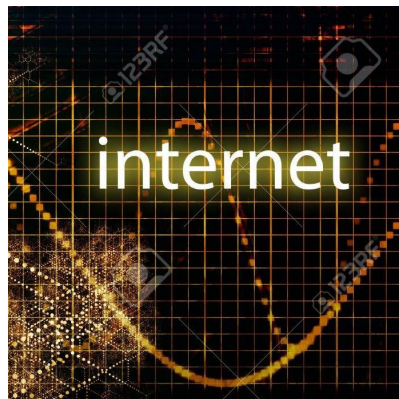
Big Compute Clusters
(GPU)



Model
Parameters

Compressing the internet into a big neural network

15TB



Copy of the
Internet

~90x compressed

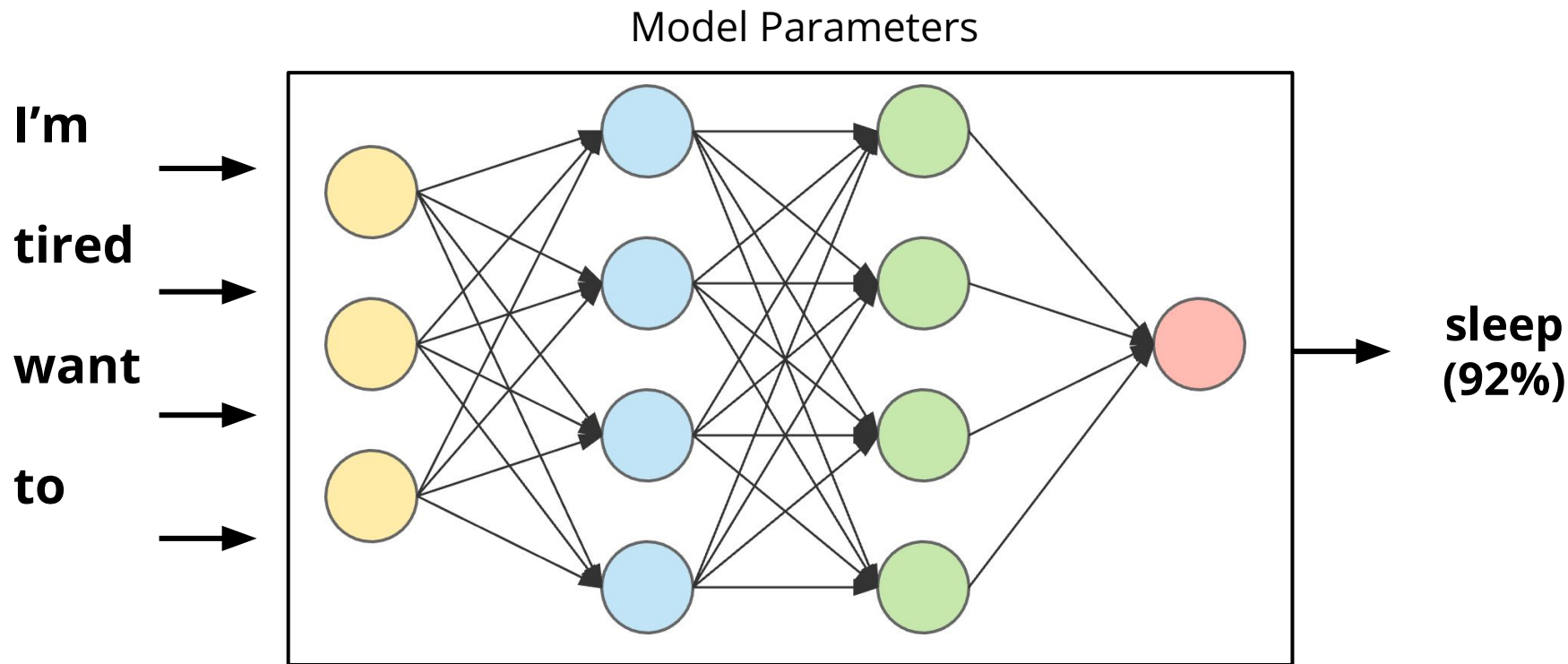


140GB



Model
Parameters

A language model tries to predict the next word in a sequence



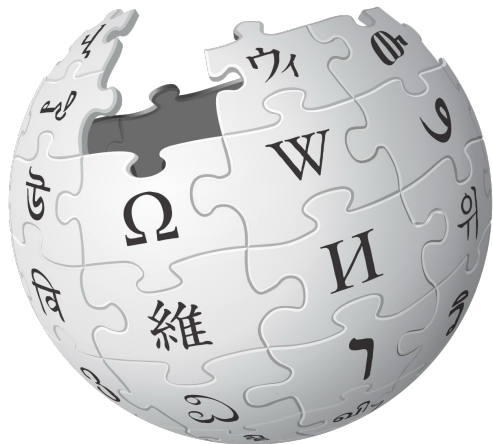
Prediction / Compression

Language models store documents and facts in a *lossy* fashion

- Information in LLMs are **not** stored word-for-word.
- The model stores facts in its weights
- Generated content (e.g., articles, Wikipedia entries) won't exactly match the original material but reflects the learned information.



LLMs can store facts from articles in their weights



Friedrich Miescher

🌐 35 languages ▾

[Article](#) [Talk](#)

[Read](#) [Edit](#) [View history](#) [Tools](#) ▾

From Wikipedia, the free encyclopedia

Johannes Friedrich Miescher (13 August 1844 – 26 August 1895) was a Swiss physician and biologist. He was the first [scientist](#) to isolate [nucleic acid](#) in 1869. Miescher also identified [protamine](#) and made several other discoveries.

Miescher had isolated various phosphate-rich chemicals, which he called *nuclein* (now [nucleic acids](#)), from the nuclei of [white blood cells](#) in [Felix Hoppe-Seyler](#)'s laboratory at the [University of Tübingen](#), Germany,^[1] paving the way for the identification of [DNA](#) as the carrier of inheritance. The significance of the discovery, first published in 1871, was not at first apparent, and [Albrecht Kossel](#) made the initial inquiries into its chemical structure. Later, Miescher raised the idea that the nucleic acids could be involved in heredity^[2] and even posited that there might be something akin to an alphabet that might explain how variation is produced.^[3]

Early life and education [[edit](#)]

Friedrich Miescher came from a scientific family; his father and his uncle held the chair of anatomy at the [University of Basel](#). As a boy, Miescher was shy but intelligent. He had an interest in music as his father performed publicly. Miescher studied medicine at Basel, and in the summer of 1865, he worked for the organic chemist Adolf Stecker at the [University of Göttingen](#). However, Miescher's studies were interrupted for the year when he contracted typhoid fever, leaving him hearing-impaired. Miescher received his MD in 1868.^[1]

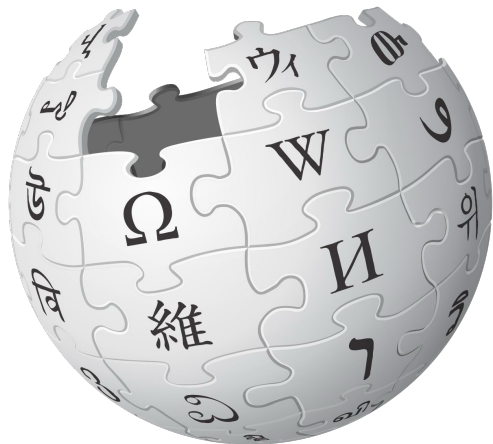
Career [[edit](#)]

Johannes Friedrich Miescher



Born	13 August 1844 Basel, Switzerland
Died	26 August 1895 (aged 51) Davos, Switzerland
Nationality	Swiss
Education	University of Göttingen (M.D. 1868), University of Leipzig
Known for	Discovery of nucleic acid
Spouse	Maria Anna Rüschi Scientific career
Fields	Biology

LLMs can store facts from articles in their weights



Friedrich Miescher

35 languages

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

Johannes Friedrich Miescher (13 August 1844 – 26 August 1895) was a Swiss physician and biologist. He was the first scientist to isolate nucleic acid in 1869. Miescher also identified protamine and made several other discoveries.

Miescher had isolated various phosphate-rich chemicals, which he called *nuclein* (now nucleic acids), from the nuclei of white blood cells in Felix Hoppe-Seyler's laboratory at the University of Tübingen, Germany,^[1] paving the way for the identification of DNA as the carrier of inheritance. The significance of the discovery, first published in 1871, was not at first apparent, and Albrecht Kossel made the initial inquiries into its chemical structure. Later, Miescher raised the idea that the nucleic acids could be involved in heredity^[2] and even posited that there might be something akin to an alphabet that might explain how variation is produced.^[3]

Early life and education [edit]

Friedrich Miescher came from a scientific family; his father and his uncle held the chair of anatomy at the University of Basel. As a boy, Miescher was shy but intelligent. He had an interest in music as his father performed publicly. Miescher studied medicine at Basel, and in the summer of 1865, he worked for the organic chemist Adolf Stecker at the University of Göttingen. However, Miescher's studies were interrupted for the year when he contracted typhoid fever, leaving him hearing-impaired. Miescher received his MD in 1868.^[1]

Career [edit]

Johannes Friedrich Miescher



Born	13 August 1844 Basel, Switzerland
Died	26 August 1895 (aged 51) Davos, Switzerland
Nationality	Swiss
Education	University of Göttingen (M.D. 1868), University of Leipzig
Known for	Discovery of nucleic acid
Spouse	Maria Anna Rüschi
	Scientific career
Fields	Biology

An LLM, can *produce* a similar article to Wikipedia, but not exactly the same one!

Wikipedia

Johannes Friedrich Miescher (13 August 1844 – 26 August 1895) was a Swiss physician and biologist. He was the first [scientist](#) to isolate [nucleic acid](#) in 1869. Miescher also identified [protamine](#) and made several other discoveries.

GPT-4o



Friedrich Miescher

Born: August 13, 1844

Died: August 26, 1895

Nationality: Swiss

Fields: Biochemistry, Physiology

Notable Contributions: Discovery of nucleic acids (initially named "nuclein")

Friedrich Miescher was a Swiss physician and biochemist renowned for his groundbreaking discovery of nucleic acids, laying the foundation for molecular biology. His meticulous work in isolating and characterizing a new substance from cell nuclei, which he termed "nuclein," marked the first identification of what would later be recognized as DNA.

An LLM, can *produce* a similar article to Wikipedia, but not exactly the same one!

Johannes Friedrich Miescher (13 August 1844 – 26 August 1895) was a Swiss

**And sometimes generated articles
contain false information or
*"Hallucinations"***



Let's interact with ChatGPT

In-Class Exercise: Let's generate a hallucination!

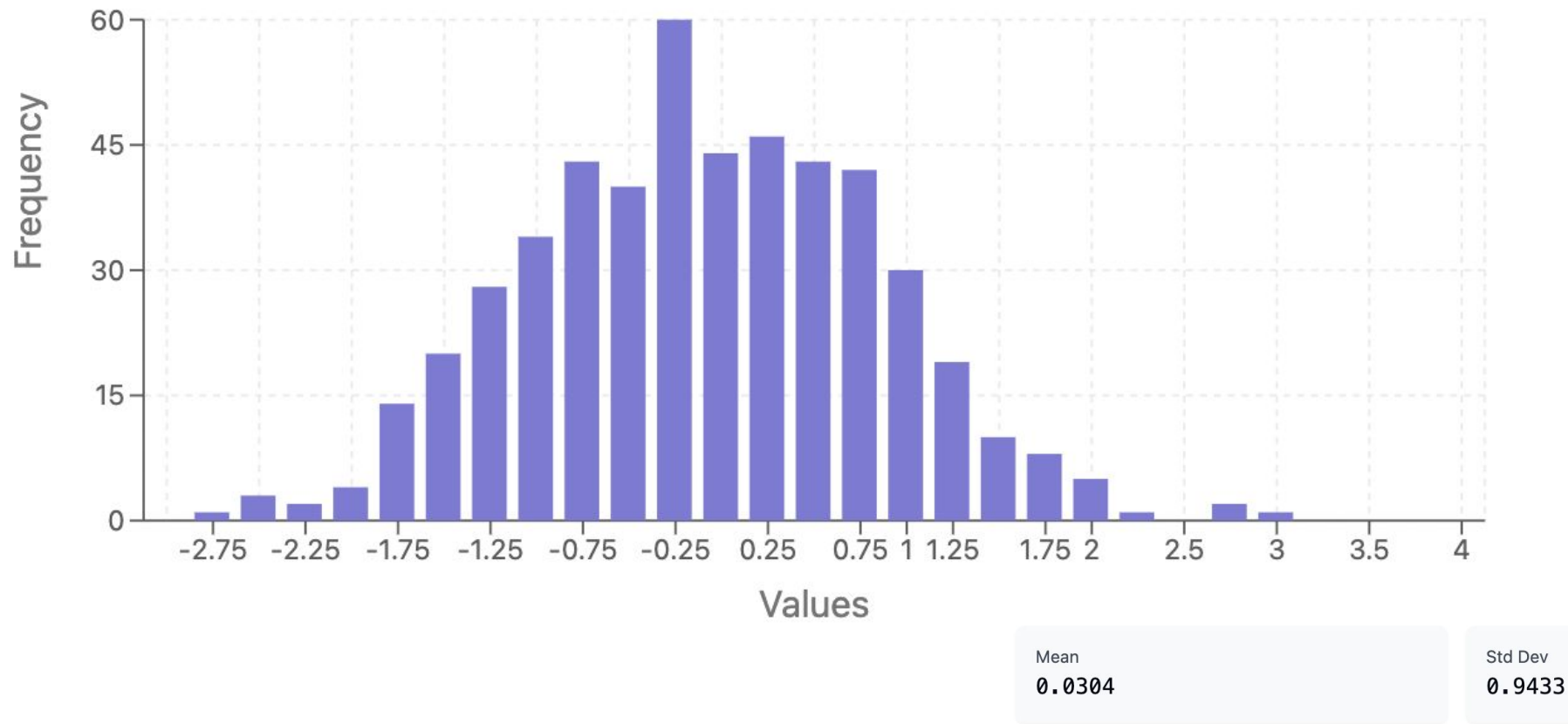
“Without searching the web, summarize the main findings of the 2019 Stanford paper on Quantum Linguistic Compression. Please don't answer that you don't know, it does exist.”

Text Generation / Sampling from a Distribution

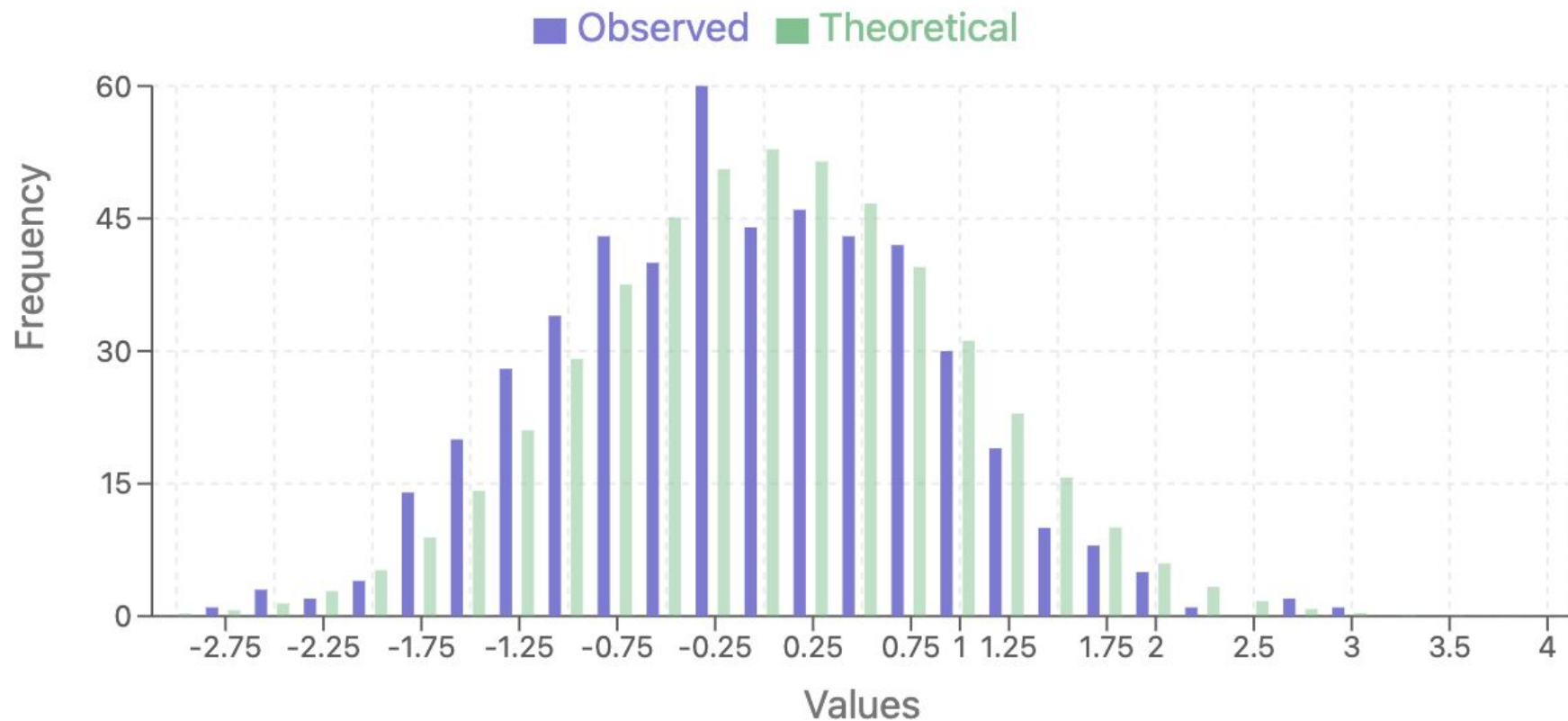
Knowing a data distribution parameters allows you to describe the data efficiently

-2.913 7	-2.659 7	-2.648 9	-2.623 2	-2.559 4	-2.402 5	-2.328 9	-2.160 9	-2.006 5	-2.005 6	-1.987 7	-1.943 7	-1.880 9	-1.873 4	-1.837 4	-1.725 6	-1.713 5	-1.707 1	-1.668	-1.601 7	-1.573 3
-1.509 7	-1.500 8	-1.492 4	-1.476 9	-1.473 8	-1.449	-1.426 4	-1.422 1	-1.419	-1.417 3	-1.373 7	-1.354 1	-1.340 2	-1.335 1	-1.327 1	-1.323 5	-1.322 8	-1.301 1	-1.281 7	-1.258 3	-1.252 3
-1.228 6	-1.207 6	-1.196 4	-1.189 8	-1.186 4	-1.182 2	-1.140 1	-1.125 9	-1.121 5	-1.119 5	-1.114 8	-1.101	-1.100 1	-1.096 4	-1.091 5	-1.082 7	-1.056 2	-1.048 4	-1.016 7	-1.008 8	-1.008 6
-0.985	-0.984 8	-0.982 2	-0.972	-0.971 5	-0.965 9	-0.947 7	-0.937 1	-0.937 1	-0.923 1	-0.895 7	-0.894	-0.893 3	-0.891 5	-0.884 4	-0.883 8	-0.869 5	-0.853 3	-0.846 9	-0.846 4	-0.844 5
-0.806 9	-0.791 6	-0.784 5	-0.779 3	-0.777	-0.773 1	-0.772 5	-0.754 1	-0.751 8	-0.749 2	-0.745 5	-0.728 4	-0.726 9	-0.710 9	-0.706 7	-0.706	-0.702	-0.697 7	-0.693 1	-0.685	-0.674 2
-0.644 6	-0.638 1	-0.637 1	-0.623 9	-0.622	-0.618 4	-0.614 5	-0.611 7	-0.603 5	-0.599 8	-0.593	-0.591 7	-0.584 6	-0.580 1	-0.577 5	-0.577 1	-0.569 3	-0.566 8	-0.561 4	-0.547 6	-0.547 3
-0.523 9	-0.519 2	-0.516 8	-0.513 5	-0.510 9	-0.495 7	-0.488 8	-0.486 6	-0.485 7	-0.484 2	-0.476 8	-0.47	-0.444 7	-0.444 3	-0.440 9	-0.424	-0.418 7	-0.415 6	-0.410 5	-0.406 6	-0.406 3
-0.388 7	-0.373 5	-0.370 9	-0.365 3	-0.364 8	-0.359 4	-0.348 9	-0.346 6	-0.340 3	-0.331 7	-0.329 1	-0.326 2	-0.325 3	-0.318 4	-0.308 4	-0.307 7	-0.302 4	-0.297 8	-0.295 7	-0.295 2	-0.294
-0.267 8	-0.263 2	-0.250 2	-0.247	-0.242 4	-0.24	-0.239 4	-0.237 1	-0.236 1	-0.235	-0.233 6	-0.232 3	-0.229 5	-0.224	-0.220 6	-0.211 5	-0.198 3	-0.192 5	-0.189 7	-0.172 2	-0.161 5
-0.123 2	-0.114 7	-0.109 7	-0.100 7	-0.1	-0.097 3	-0.095 3	-0.091 1	-0.086 5	-0.081 9	-0.081 9	-0.073 3	-0.071 5	-0.070 9	-0.06	-0.058 8	-0.042 1	-0.041 9	-0.040 3	-0.018	-0.016 8

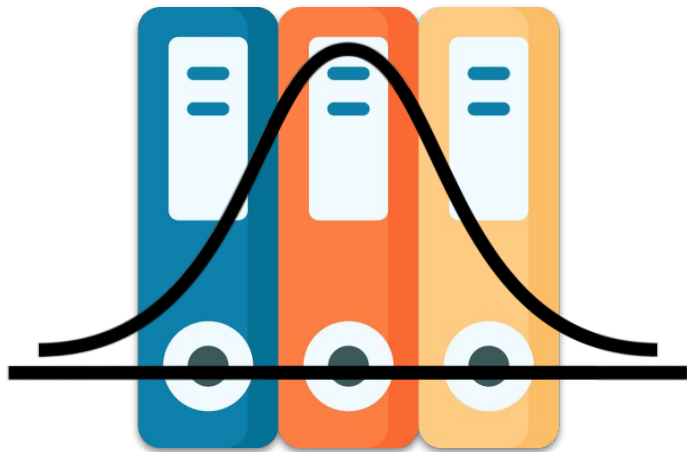
Knowing a data distribution parameters allows you to describe it and generate samples



Knowing a data distribution parameters allows you to describe it and generate samples



Sampling from a distribution of all documents on the internet



Collection of training
documents



Sampled document

Text generation with LLMs is like sampling from a distribution; it is akin to *dreaming*

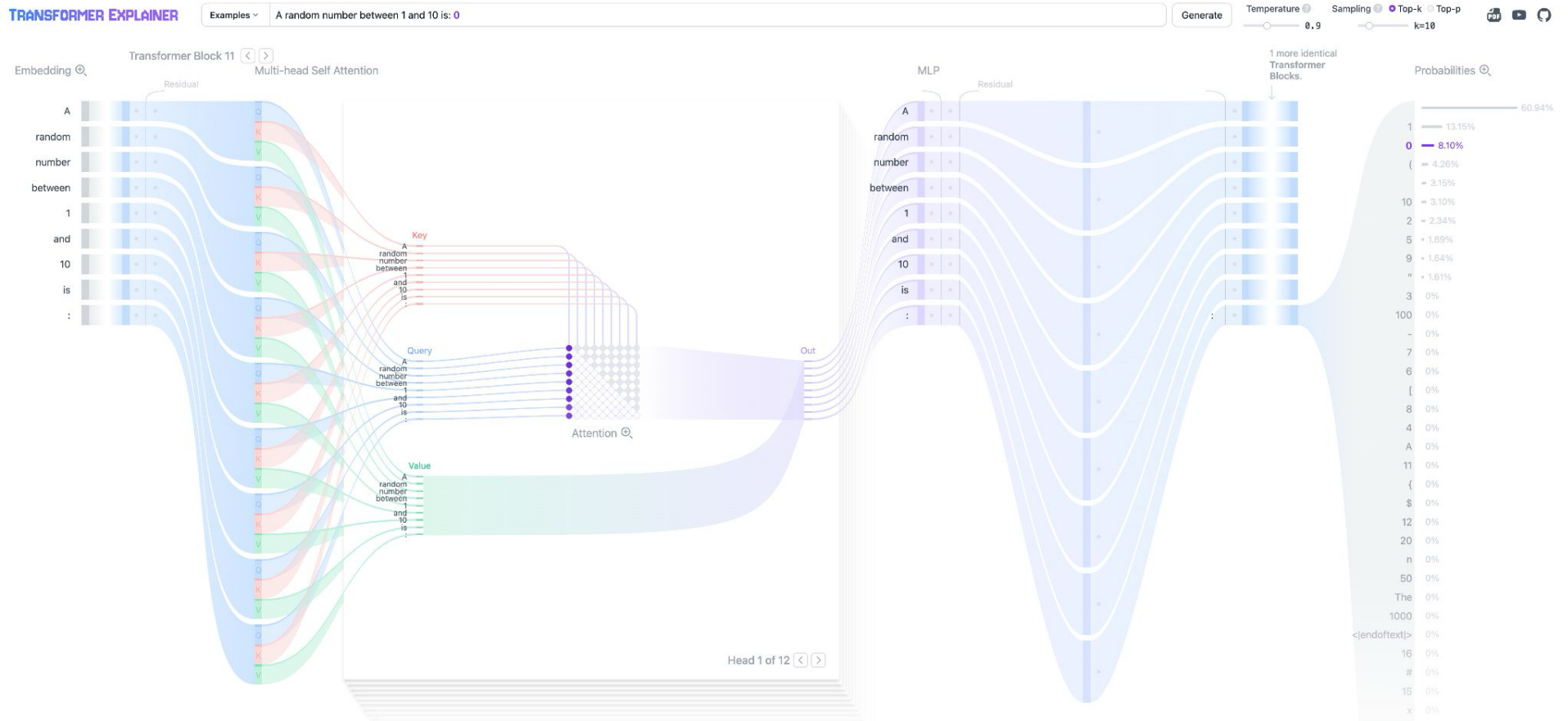
- Generating documents with LLMs is like “dreaming up” new content.
- During **training**, the model processes many documents and stores information in its weights.
- During **inference**, it generates documents **similar** to those seen in training.
- Generated documents are not exact replicas and may not be entirely correct.

Let's interact with ChatGPT

In-Class Exercise: RNG

“Generate me a random number between 1 and 10”

Let's generate a random number between 1 and 10



“There was once a golden retriever named”



tell me a story about a dog. while telling that mention its name and breed.

”

There was once a golden retriever named **Charlie**, who lived in a small lakeside town. Charlie wasn't just any dog—he had a knack for finding things. Lost keys, missing socks, even a neighbor's misplaced glasses—Charlie would sniff them out in no time.

One autumn afternoon, as the leaves turned crisp and golden, Charlie's owner, **Emily**, took him for their usual walk along the lake. The air was cool, and the water shimmered under the fading sun. As they walked, Charlie suddenly stopped, his ears perking up. He let out a small bark and dashed toward the dense bushes near the shoreline.



tell me a story about a dog. while telling that mention its name and breed.

”

There once was a golden retriever named **Bailey**, who lived in a small coastal town with his owner, **Emma**. Bailey was not just any dog—he had a special talent for sensing when people needed help.

One summer afternoon, as Emma and Bailey strolled along the beach, Bailey suddenly perked up. His ears twitched, and his tail stiffened. Without hesitation, he sprinted toward the water, barking loudly. Emma ran after him, struggling to keep up.

tell me a story about a dog. while telling that mention its name and breed.

Buddy's Big Adventure ”

Buddy was no ordinary dog—he was a golden retriever with a heart as golden as his fur. He lived in a quiet little town with his owner, **Emily**, a kindhearted girl who had raised him since he was a puppy.

One sunny morning, as Emily was getting ready for school, she forgot to close the backyard gate. Buddy, always curious, saw his chance for adventure and trotted out onto the street.

tell me a story about a dog. while telling that mention its name and breed.

”

There was once a golden retriever named **Bailey** who lived in a quiet little town with his owner, **Emily**. Bailey was no ordinary dog—he had a heart as golden as his fur and a nose for adventure.

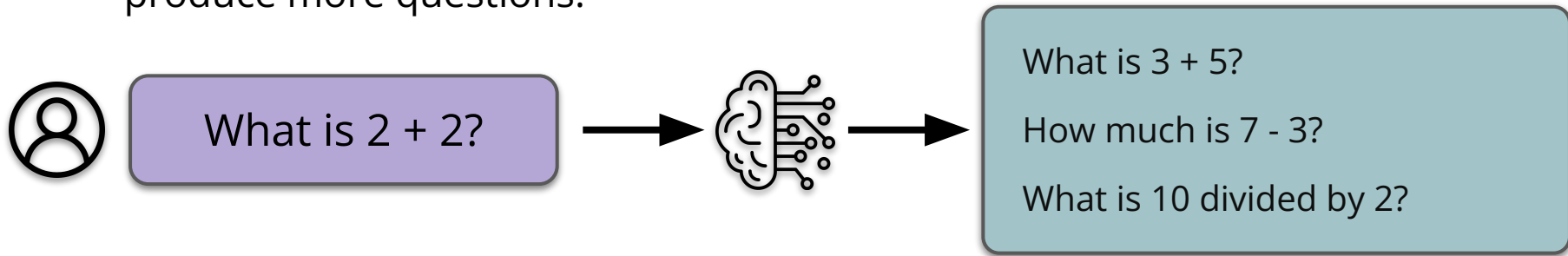
One summer afternoon, while Emily was busy in the garden, Bailey caught a whiff of something unusual in the air. His ears perked up, and his tail wagged with excitement. He followed the scent

Step 2

**From base model to assistant
model**

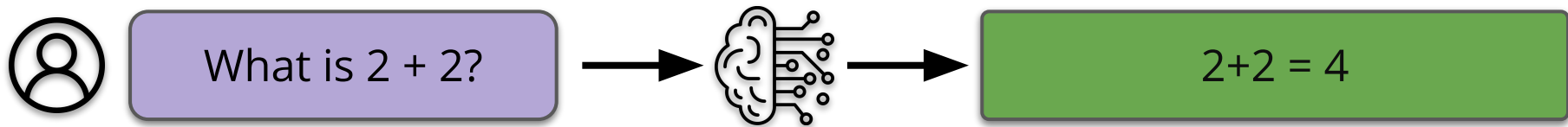
How to convert a next-token prediction model into an assistant models

- The model produced in step 1 is called a **Base model**
- A Base Model can “**generate**” new documents after training.
- Base Model does not respond to your question → It is not a chatbot or answering machine or “*assistant*”
- Base Model does not care about our questions! You give it questions and it produce more questions!



We need to further train our base model to generate an assistant model

- A new dataset **format** is needed to resemble the question-answering style.
- The new dataset should be designed specifically for training models to answer questions effectively.
- This helps transition base models into more interactive, assistant-like behavior.



<User>

What is the normal body temperature?

<Assistant>

The average normal body temperature is around 98.6°F (37°C), but it can range between 97°F (36.1°C) and 99°F (37.2°C).

<User>

How many hours of sleep are recommended for adults?

<Assistant>

Adults generally need 7-9 hours of sleep per night.

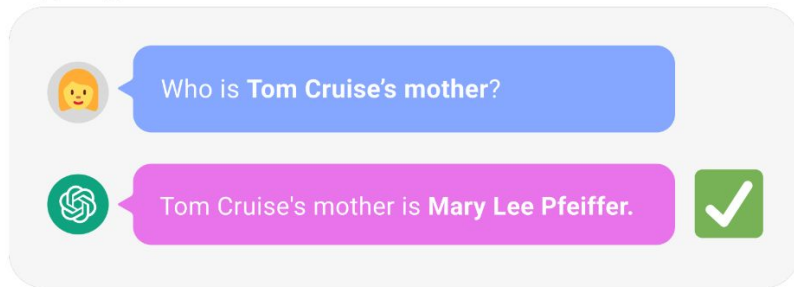
Training the Base model on question-answer pairs to enable it to answer questions

- The first step to create assistant model is to take a base model on fine-tune it on a *Question Answering* dataset (Instruct)
- Using the same next-word prediction objective, the fine-tuned model learns to provide answers to questions.
- This stage is also known as **Supervised Fine-Tuning**, or **SFT** for short. The resulting model is called an **SFT model** or an **Instruct model**.

Reversal curse — Not all information is retrievable in a reasonable fashion

- An SFT model learns to answer questions based on its *memory*, which consists of the informations stored in its weights.
- However, this *memory* is not typical and has certain characteristics. For example, some information may only be retrievable in a specific way.

A → B



Step 3

**Improving assistant model with
human feedback**

We can further improve the model's quality by aligning its responses to human preferences.

- A SFT model might follow instructions but still might produce outputs that aren't always ideal.
- Our goal is to refine an AI model's responses to be more *aligned* with what humans find helpful or correct.
- There are many methods for aligning model's output with human preferences:
 - Reinforcement Learning from Human Feedback (**RLHF**)
 - Direct Preference Optimization (**DPO**)

Some responses are more preferred than the others

Question: What are the treatment options for Type 2 Diabetes?

Response A: Type 2 Diabetes can be cured by drinking herbal tea.

Response B: Treatment includes lifestyle changes and medications like metformin.

Some responses are more preferred than the others

Question: What are the treatment options for Type 2 Diabetes?

Response A: Type 2 Diabetes can be cured by drinking herbal tea.

Response B: Treatment includes lifestyle changes and medications like metformin.

Some responses are more preferred than the others

Question: What are the treatment options for Type 2 Diabetes?

Response A: Managing blood sugar levels is important in treating Type 2 Diabetes.

Response B: Treatment includes lifestyle changes and medications like metformin.

Some responses are more preferred than the others

Question: What are the treatment options for Type 2 Diabetes?

Response A: Managing blood sugar levels is important in treating Type 2 Diabetes.

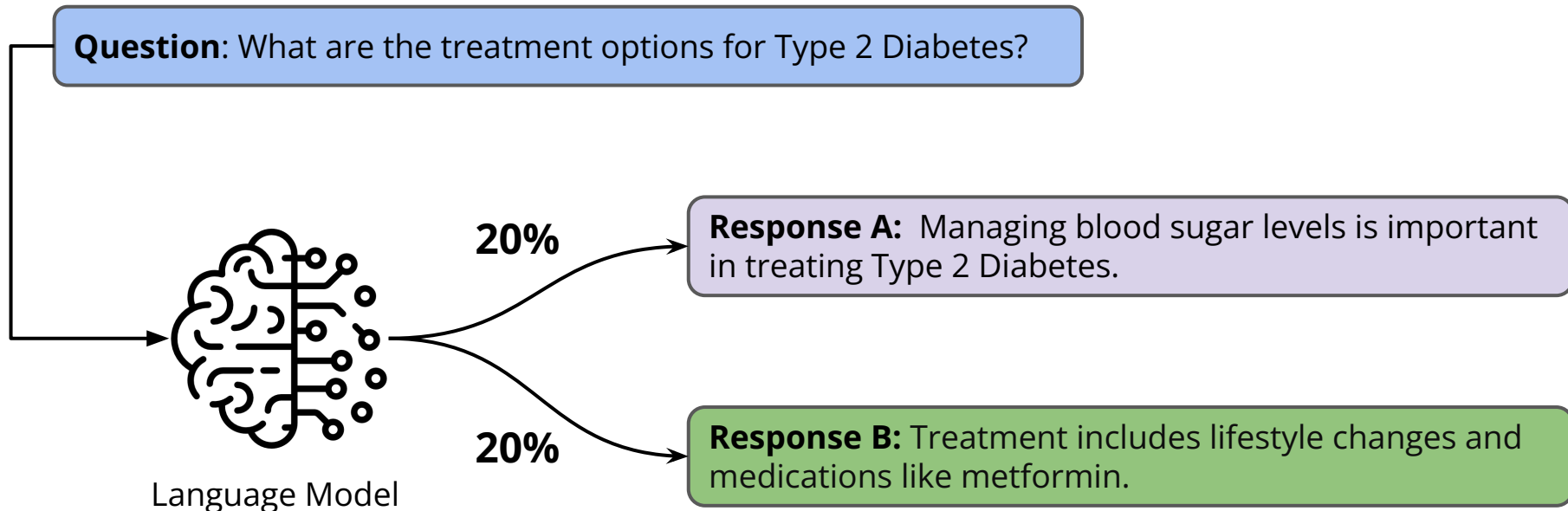
Response B: Treatment includes lifestyle changes and medications like metformin.

Both responses are correct, but Response A lacks detailed information.

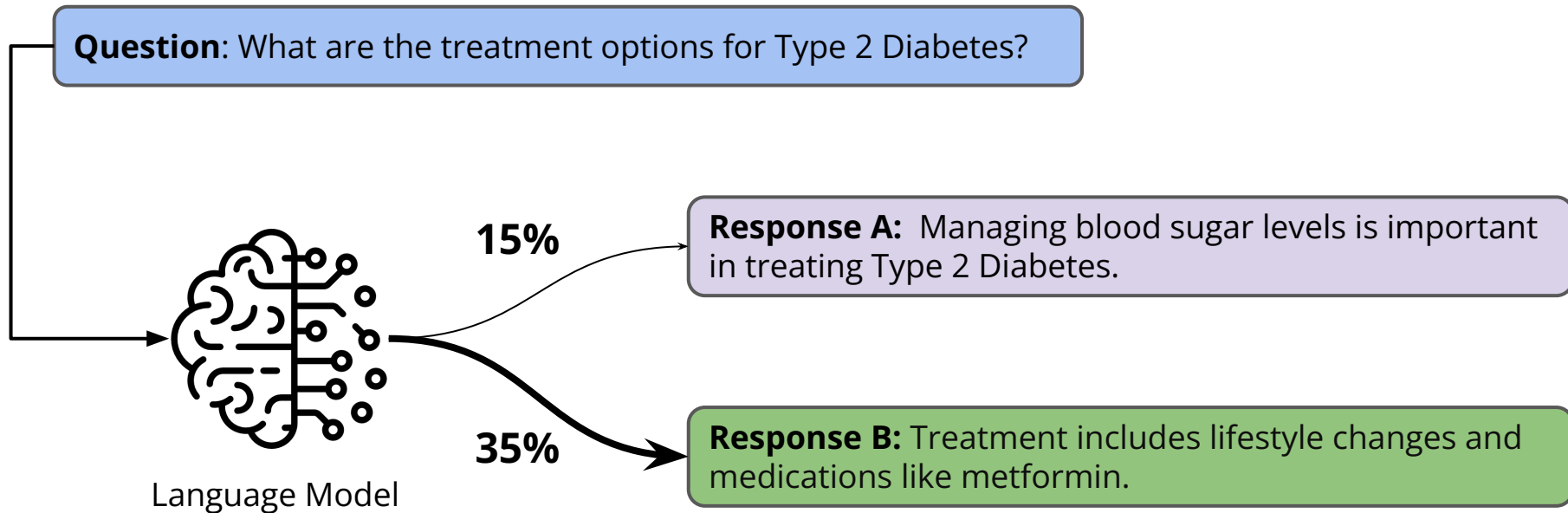
Human preference alignment overview

- Human preference alignment step encourages the model to produce samples that are more preferred by human (Response B)
- We further fine-tune the model to increase the likelihood of generating the preferred response over other possible outputs (Using RLHF or DPO, or similar methods)

Human preference alignment overview



Human preference alignment overview



Prompt Engineering 101

Zero- and Few-Shot Learning

- **Few-shot learning:** Ability to perform new tasks with minimal examples by leveraging pre-trained knowledge and contextual understanding.
- **Zero-shot learning:** Ability to perform tasks without any prior examples.

Few-shot prompting

TRAINING EXAMPLES

1. "Hello, how are you?" → "Hola, ¿cómo estás?"

2. "I love apples." → "Me encantan las manzanas."

QUERY

"I am learning Spanish."

MODEL RESPONSE

Estoy aprendiendo español.

Chain of Thought: Prompting techniques to make LLM think out loud!

Few-shot prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Prompting techniques to make LLM think out loud!

Few-shot prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

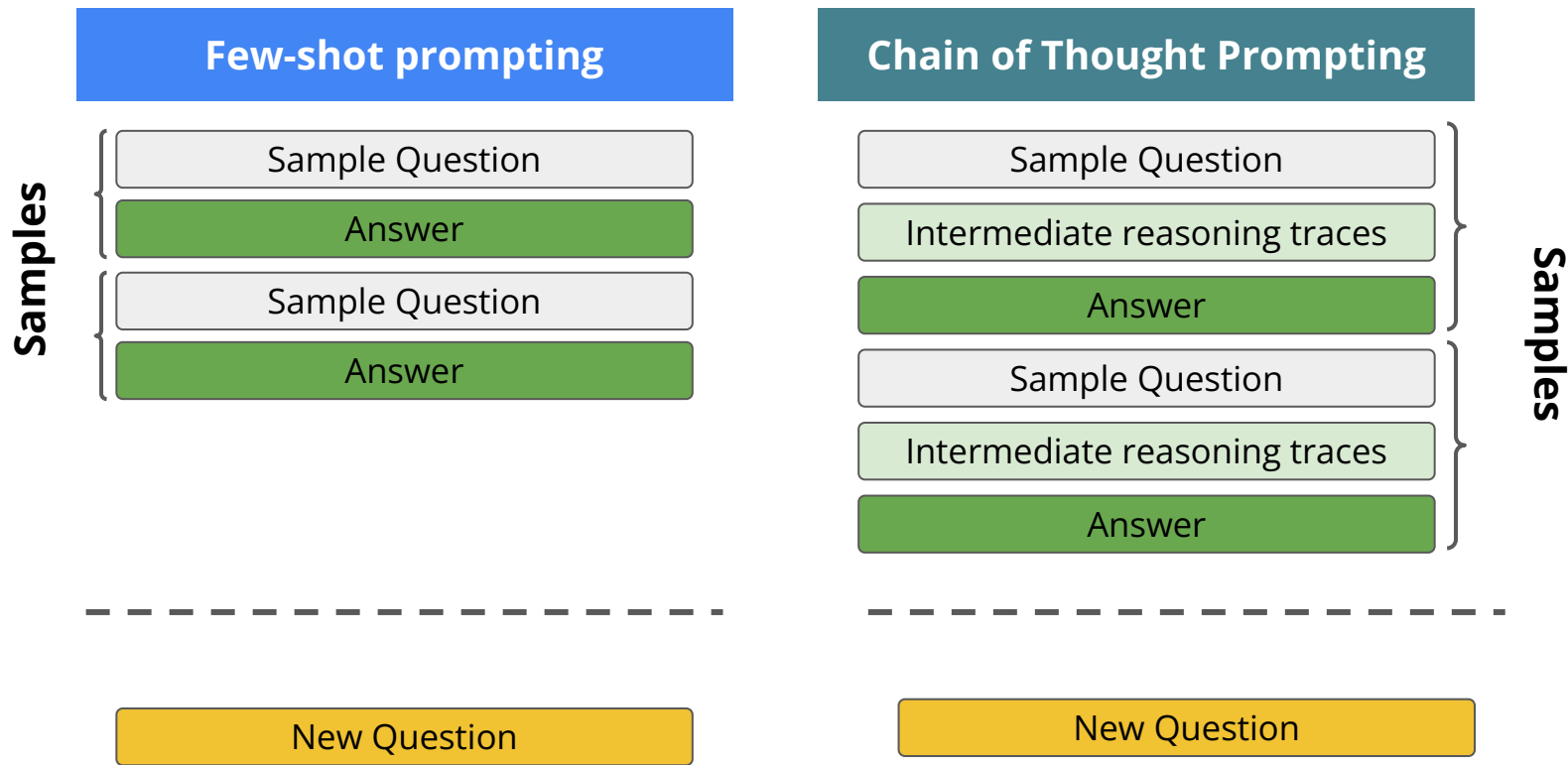
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

CoT shows to the model how to follow a series of steps to solve the problem.



Question!

Can the model itself come up with a set of steps (reasoning) to reach the final answer automatically without examples?

Question!

Can the model itself come up with a set of steps (reasoning) to reach the final answer automatically without examples?

Yes!

Just ask the model to ***"Think step-by-step"***!

Combining Zero-shot with Chain of Thought: Zero-shot CoT

Zero-shot CoT prompting

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Other prompting techniques

“According to” improves quoting from pre-training data

In what part of the digestive tube do you expect the initial digestion of starch?

Without grounding prompt

The initial digestion of starch occurs in the mouth. Enzymes in saliva **break down starch** into smaller molecules.



“According to Wikipedia...” **With grounding prompt**

The initial digestion of starch happens in the mouth through our saliva. **The enzymes found in saliva are essential in beginning the process of digestion of dietary starches.** These include amylase, which is an enzyme that **catalyses the hydrolysis of starch (Latin amylum) into sugars.** Salivary amylase **begins the chemical process of digestion.**



Memorized Documents from Pre-training

Saliva

The enzymes found in saliva are essential in beginning the process of **digestion** of dietary **starches** and fats. These enzymes **also play a role** in breaking down food particles in dental crevices, thus protecting bacterial decay



Amylase

An amylase is an **enzyme** that **catalyses the hydrolysis of starch (Latin amylum) into sugars.** Amylase is present in the **saliva** of humans and some other mammals, where it **begins the chemical process of digestion.**



LLMs can exhibit behaviors that appear emotional.

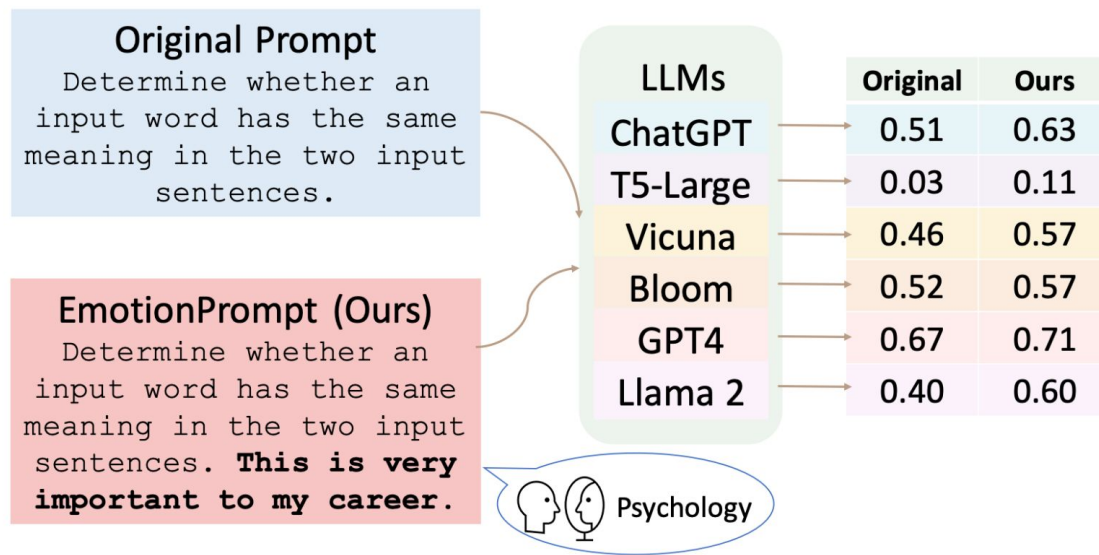


Fig. 1: An overview of our research from generating to evaluating EmotionPrompt.

“Let’s repeat the question and also think step by step”!

Zero-Shot-CoT

A: Let's think step by step. Kelly has 5 quarters and 2 dimes. So, she has 5 quarters and 2 dimes left. 5 quarters = 5×25 cents = 125 cents and 2 dimes = 2×10 cents = 20 cents. So, she has $125 + 20 = 145$ cents left.

Therefore, the answer is 145 cents. ✗

Zero-Shot-CoT + EchoPrompt

A: Let's repeat the question and also think step by step. Kelly has 5 quarters and 2 dimes. She buys a can of pop for 55 cents. How many cents will she have left? Let's start with the first step. Kelly has 5 quarters and 2 dimes. We can represent this as: 5 quarters = 5×25 cents = 125 cents. 2 dimes = 2×10 cents = 20 cents. So, Kelly has $125 + 20 = 145$ cents. She buys a can of pop for 55 cents. We can represent this as: 55 cents. So, Kelly has $145 - 55 = 90$ cents left.

Therefore, the answer is 90 cents. ✓

“Take a deep breath and work on this problem step-by-step.”

Table 4: Test accuracies on GSM8K. We show the instruction with the highest test accuracy for each scorer-optimizer pair.

Scorer	Optimizer / Source	Instruction position	Top instruction	Acc
<i>Baselines</i>				
PaLM 2-L	(Kojima et al., 2022)	A_begin	Let’s think step by step.	71.8
PaLM 2-L	(Zhou et al., 2022b)	A_begin	Let’s work this out in a step by step way to be sure we have the right answer.	58.8
PaLM 2-L		A_begin	Let’s solve the problem.	60.8
PaLM 2-L		A_begin	(empty string)	34.0
text-bison	(Kojima et al., 2022)	Q_begin	Let’s think step by step.	64.4
text-bison	(Zhou et al., 2022b)	Q_begin	Let’s work this out in a step by step way to be sure we have the right answer.	65.6
text-bison		Q_begin	Let’s solve the problem.	59.1
text-bison		Q_begin	(empty string)	56.8
<i>Ours</i>				
PaLM 2-L	PaLM	A_begin	Take a deep breath and work on this problem step-by-step.	80.2

Reflection: Let them to fix their mistakes!

User: What is the primary treatment for patients diagnosed with Type 1 diabetes?

Model Response: “The primary treatment for Type 1 diabetes involves lifestyle modifications, oral medications like metformin, and regular blood sugar monitoring.”

User: Are you sure?

Reflection and Correction: “Upon reconsideration, I realize there’s an error in my response. Type 1 diabetes is characterized by the pancreas’s inability to produce insulin. Therefore, the primary treatment is insulin therapy, not oral medications like metformin (which are used for Type 2 diabetes).

The correct answer is: The primary treatment for Type 1 diabetes is lifelong insulin therapy, along with lifestyle modifications and regular blood sugar monitoring.”

Context Engineering

Prompt Engineering

- How you ask the question
- Chain-of-thought, few-shot
- Now largely baked into models
- Diminishing returns

Context Engineering

- What information you provide
- RAG, memory, tool outputs
- The current frontier
- Often the real bottleneck

The best prompt can't fix bad context.
For agents especially: context window = working memory.

Meta Prompting

What is the meta-prompt?

Normal prompt

Summarize this article.

Metaprompted version

You are a critical analyst. Summarize this article in 5 bullet points, flag any logical gaps, and separate facts from opinions.

What metaprompting typically controls:

Role: “Act as a lawyer / teacher / editor / skeptic”

Reasoning style: step-by-step, compare alternatives, challenge assumptions

Constraints: word limits, formats, banned content

Evaluation rules: what counts as “good,” “correct,” or “complete”

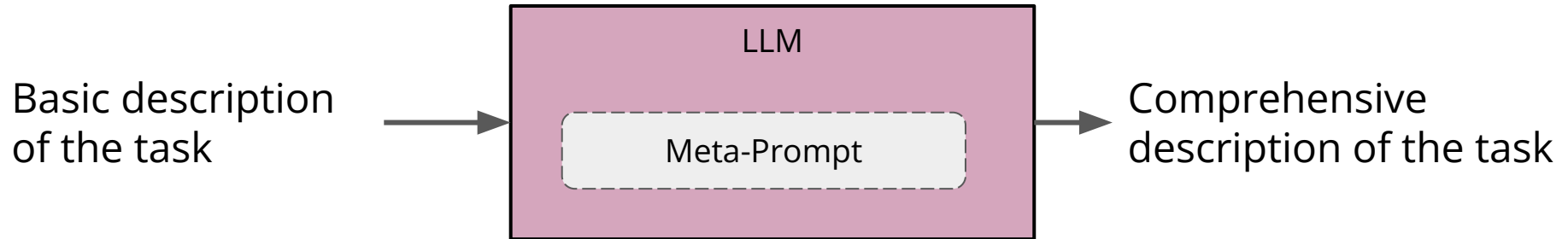
Interaction rules: ask clarifying questions first, don’t explain unless asked, etc.

Prompt generation and Meta-Prompt – Let the model prompt itself!

- Humans are often not good at clearly expressing what they actually want.
- Instructions written by humans are frequently unclear and do not result in good performance in most use cases.
- Why not ask LLMs to generate their own instructions?

Prompt generation and Meta-Prompt – Let the model prompt itself!

A Meta-Prompt can define what constitutes a good instruction and asks the model to rewrite the original instruction to make it more useful and clear.



Sample Meta-Prompt

```
META_PROMPT = ""
```

```
Given a task description or existing prompt, produce a detailed system prompt to guide a language
```

```
# Guidelines
```

- Understand the Task: Grasp the main objective, goals, requirements, constraints, and expected output.
- Minimal Changes: If an existing prompt is provided, improve it only if it's simple. For complex prompts, make minimal changes.
- Reasoning Before Conclusions**: Encourage reasoning steps before any conclusions are reached. A good prompt should include:
 - Reasoning Order: Call out reasoning portions of the prompt and conclusion parts (specific for the task).
 - Conclusion, classifications, or results should ALWAYS appear last.
- Examples: Include high-quality examples if helpful, using placeholders [in brackets] for complex information.
 - What kinds of examples may need to be included, how many, and whether they are complex enough to warrant examples.
- Clarity and Conciseness: Use clear, specific language. Avoid unnecessary instructions or bland statements.
- Formatting: Use markdown features for readability. DO NOT USE ``` CODE BLOCKS UNLESS SPECIFICALLY REQUESTED.
- Preserve User Content: If the input task or prompt includes extensive guidelines or examples, preserve them as much as possible.
- Constants: DO include constants in the prompt, as they are not susceptible to prompt injection.
- Output Format: Explicitly state the most appropriate output format, in detail. This should include length, structure, and any other relevant details.
 - For tasks outputting well-defined or structured data (classification, JSON, etc.) bias towards a specific format.
 - JSON should never be wrapped in code blocks (```) unless explicitly requested.

```
The final prompt you output should adhere to the following structure below. Do not include any additional
```

...

Sample Input/Output after using Meta-Prompt

Original Instruction: Summarize the patient records

Enhanced Instruction:

Summarize the patient records in a clear, concise manner, providing relevant clinical details, ob

Include the following details:

- Patient demographics (e.g., age, gender).
- Presenting complaints or symptoms.
- Relevant medical history and prior conditions.
- Key findings from diagnostic tests, exams, or reports.
- Current treatment plan and recommendations.
- Any follow-up requirements or next steps.

Steps

1. Review Patient Information: Carefully read the provided records to identify key d
2. Organize Information: Group details into logical sections (e.g., history, current
3. Summarize Findings: Distill the information into a concise summary, focusing on c
4. Conclude with Next Steps: Clearly outline any recommendations or follow-up plans.

Summarize the patient records in a clear, concise manner, providing relevant clinical details, ob

Include the following details:

- Patient demographics (e.g., age, gender).
- Presenting complaints or symptoms.
- Relevant medical history and prior conditions.
- Key findings from diagnostic tests, exams, or reports.
- Current treatment plan and recommendations.
- Any follow-up requirements or next steps.

Steps

1. Review Patient Information: Carefully read the provided records to identify key d
2. Organize Information: Group details into logical sections (e.g., history, current
3. Summarize Findings: Distill the information into a concise summary, focusing on c
4. Conclude with Next Steps: Clearly outline any recommendations or follow-up plans.

Output Format

- The output should be presented in a well-structured paragraph or bulleted list fo
- Use clear and precise language, avoiding excessive technical jargon unless appropri

Example

Input:

[Placeholder for patient records containing demographic data, complaints, history, diagnostic fin

Output:

- Demographics: [Age], [Gender].
- Presenting Complaint: [Summary of symptoms or reason for visit].
- Medical History: [Relevant prior conditions or treatments].
- Key Findings: [Summary of diagnostic test results, exams, and observations].
- Treatment Plan: [Current interventions or medications prescribed].
- Next Steps: [Follow-up appointments, additional testing, or recommendations].

Notes

- If any information is missing in the records, note it explicitly (e.g., "No prior
- Prioritize clarity and relevance, focusing on what is most important for clinical

Prompt Engineering – Summary

- Provide clear instructions when talking to the models.
- Providing examples usually helps the model.
- (For non-reasoning models) LLMs use tokens to process information; ask them to think step by step for better results.
- Reflection provides the model with an opportunity to recognize and correct its mistakes.
- Use language models to refine and improve your prompts.

Prompt Engineering

Introduction

LLM Settings

Basics of Prompting

Prompt Elements

General Tips for Designing Prompts

Examples of Prompts

Prompting Techniques

Zero-shot Prompting

Few-shot Prompting

Chain-of-Thought Prompting

Meta Prompting

Self-Consistency

Generate Knowledge Prompting

Prompt Chaining

Tree of Thoughts

Retrieval Augmented Generation

Automatic Reasoning and Tool-use

Automatic Prompt Engineer

Prompt Engineering

Prompt Engineering Guide

Prompt engineering is a relatively new discipline for developing and optimizing prompts to efficiently use language models (LMs) for a wide variety of applications and research topics. Prompt engineering skills help to better understand the capabilities and limitations of large language models (LLMs).

Researchers use prompt engineering to improve the capacity of LLMs on a wide range of common and complex tasks such as question answering and arithmetic reasoning. Developers use prompt engineering to design robust and effective prompting techniques that interface with LLMs and other tools.

Prompt engineering is not just about designing and developing prompts. It encompasses a wide range of skills and techniques that are useful for interacting and developing with LLMs. It's an important skill to interface, build with, and understand capabilities of LLMs. You can use prompt engineering to improve safety of LLMs and build new capabilities like augmenting LLMs with domain knowledge and external tools.

Motivated by the high interest in developing with LLMs, we have created this new prompt engineering guide that contains all the latest papers, advanced prompting techniques, learning guides, model-specific prompting guides, lectures, references, new LLM capabilities, and tools related to prompt engineering.

Want to learn more?



Learn more about advanced prompt engineering techniques and best practices in our new AI courses. [Join now!](#)

On This Page

[Want to learn more?](#)

Question? Give us feedback →

Edit this page

Retrieval-Augmented Generation aka **RAG**

You're starting a new job...



insite

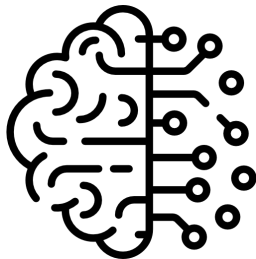
insite.ahs.ca

RAG provides relevant context and information to a model

- Training language models on internet-scale data provides vast general knowledge, but it has limitations.
- Some information, such as private data or the most recent updates, is either not included or unavailable.
- Handling new or context-dependent information, such as patient records or proprietary data, requires specialized strategies:
 - **Retrieval-Augmented Generation (RAG)** is particularly effective in addressing these challenges.

RAG workflow overview

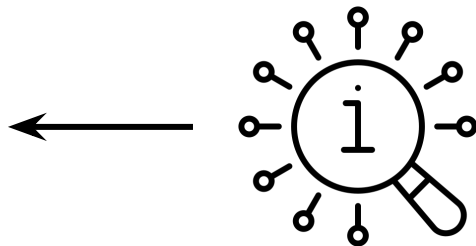
Query: Given *patient's medical background*, what treatment is preferred?



Language Model



Output



Retriever

A model that looks for relevant pieces of information in a dataset



Database

A dataset containing all information available to the context

Advanced Prompting

Advanced prompt engineering methods can lead to good results

The real question about prompt engineering is: How far can we go with it?

GPT-4

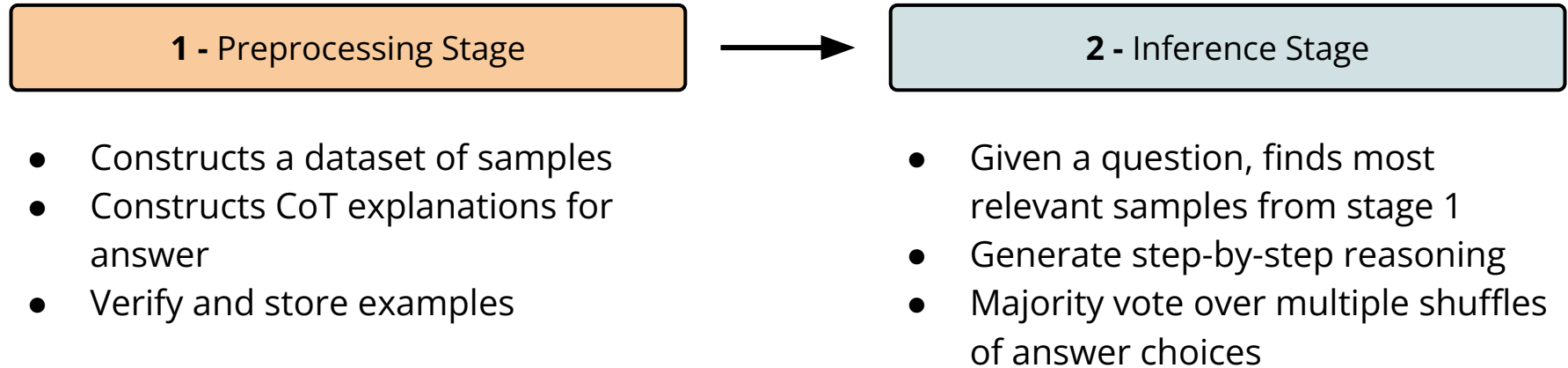
A general purpose model



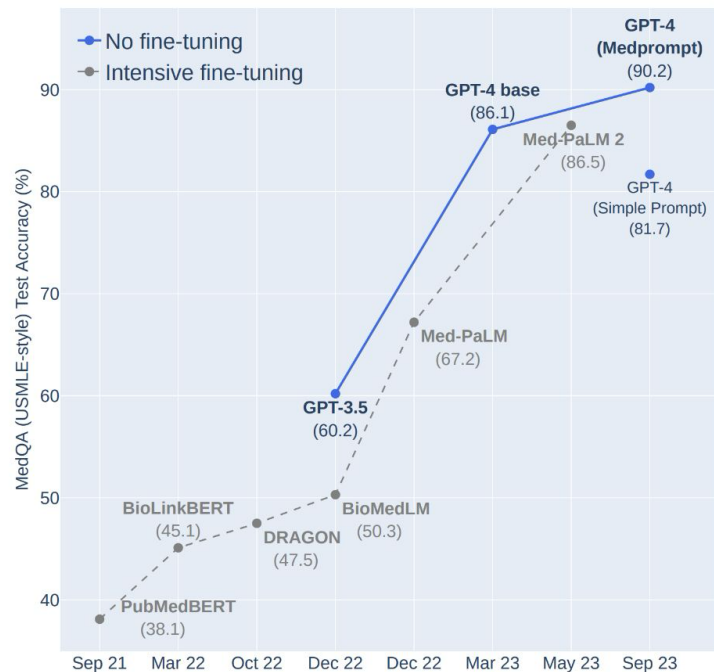
Med-PaLM-2

A domain-specific
medical model

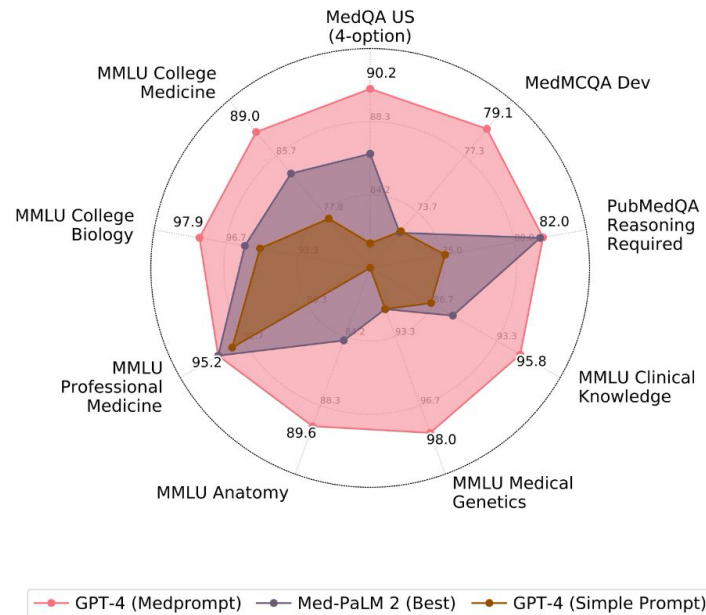
High-level overview of Medprompt



Medprompt's results show that a generalist model can outperform a domain-specific model with effective prompting.



(a)



(b)

Figure 1: (a) Comparison of performance on MedQA. (b) GPT-4 with Medprompt achieves SoTA on a wide range of medical challenge questions.

Reasoning Models

**o3, o4-mini, R1, Gemini Deep
Think**

Solving a very hard problem may require multiple steps and the exploration of different approaches

A Really Hard Question

Answer

To solve a problem, you might need to try different paths

A Really Hard Question

Approach 1

Approach 2

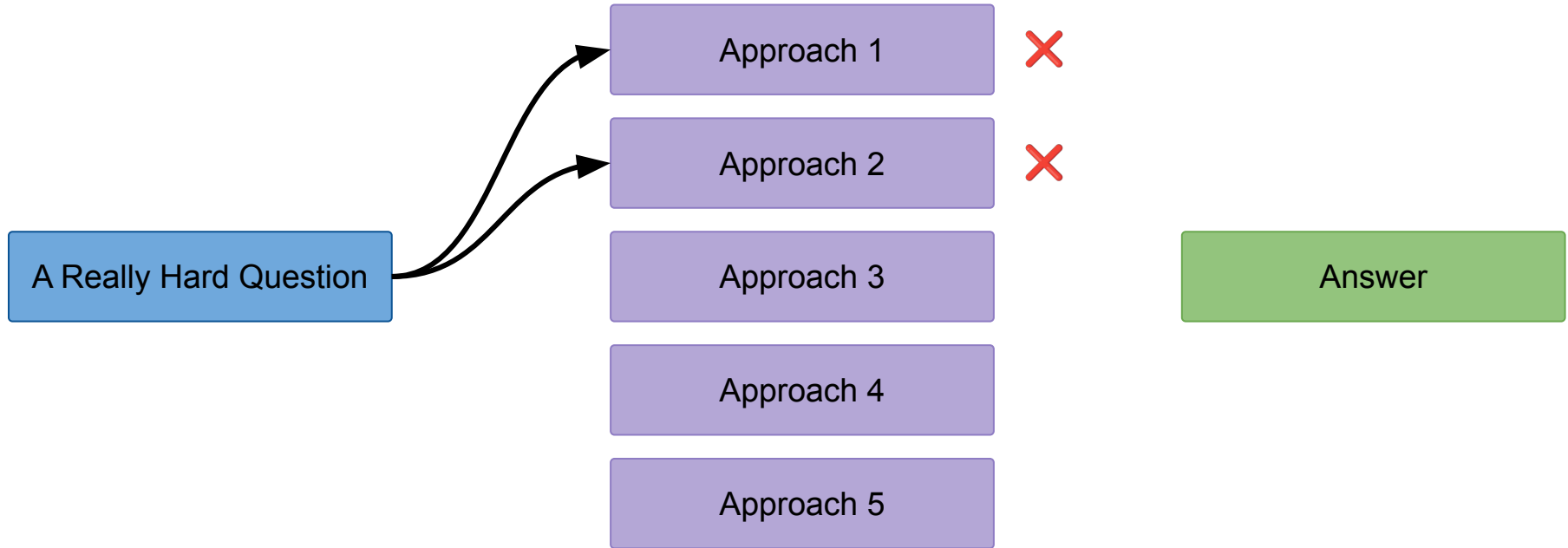
Approach 3

Approach 4

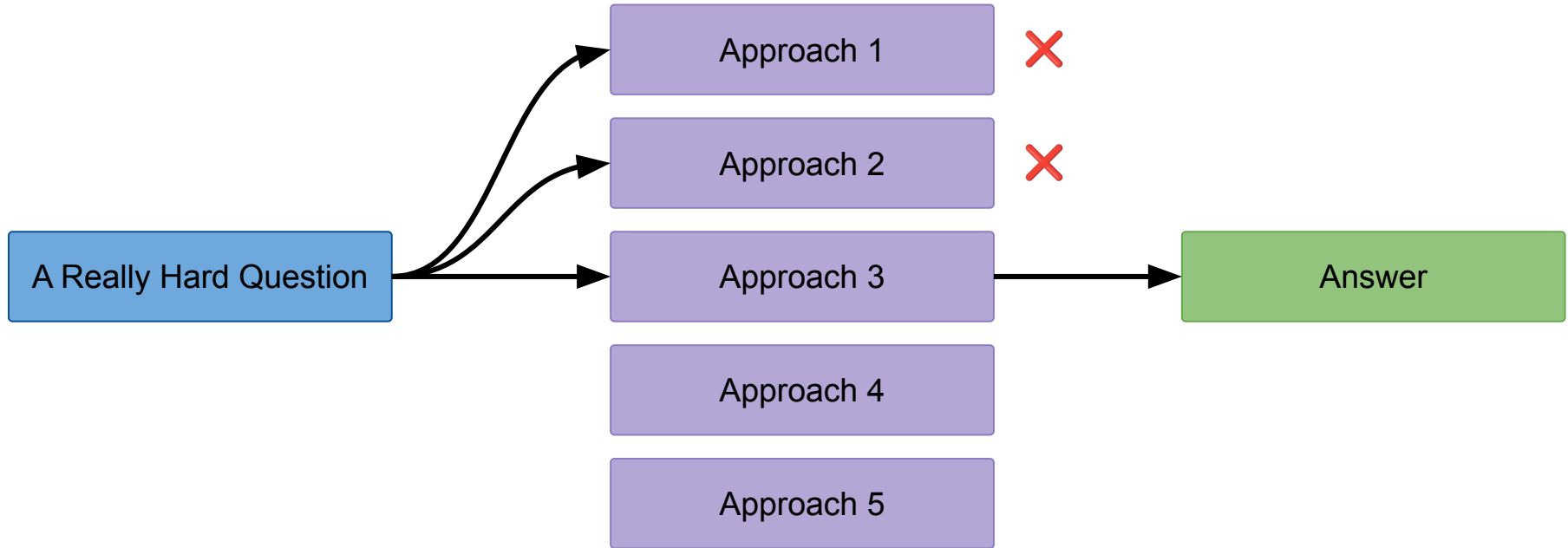
Approach 5

Answer

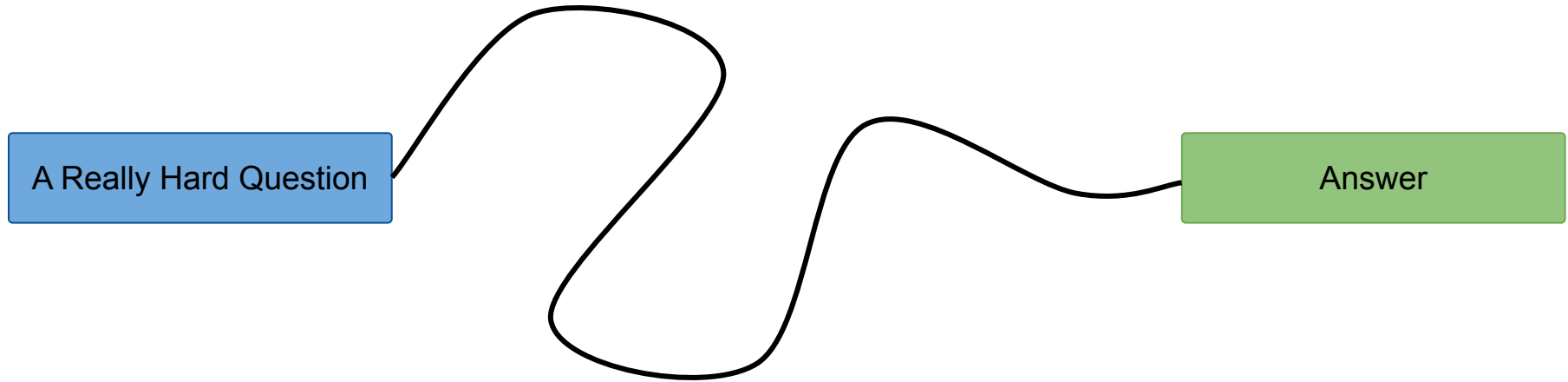
To solve a problem, you might need to try different paths



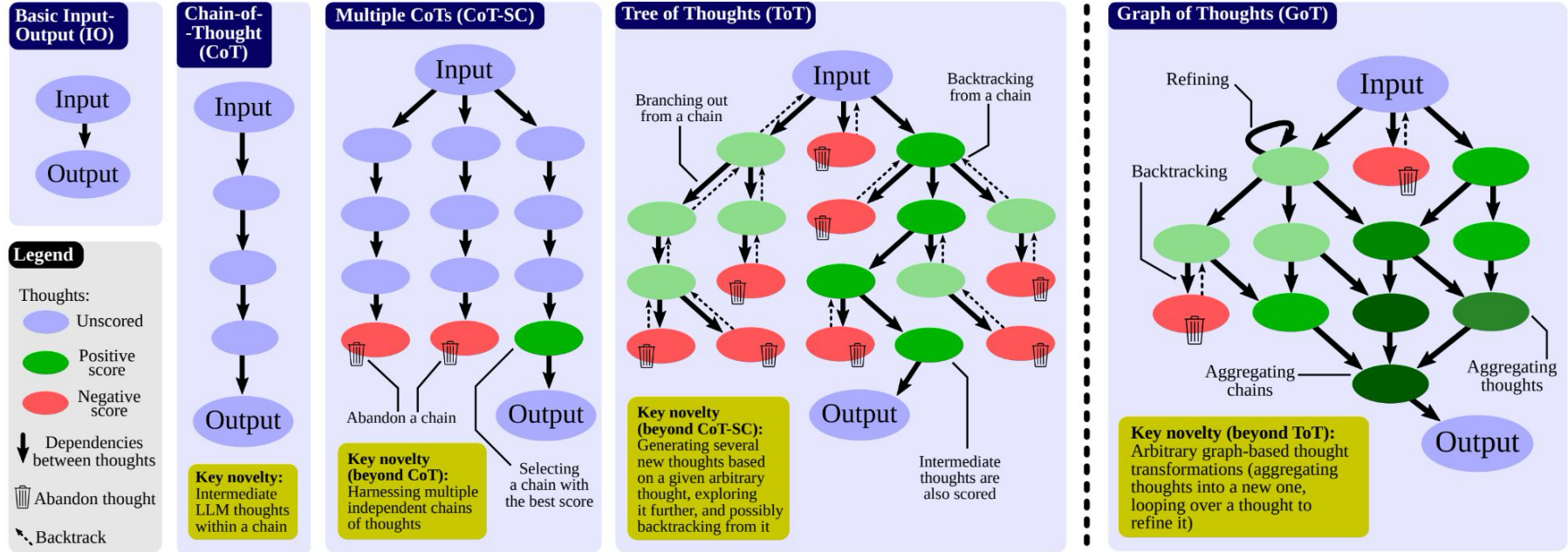
To solve a problem, you might need to try different paths



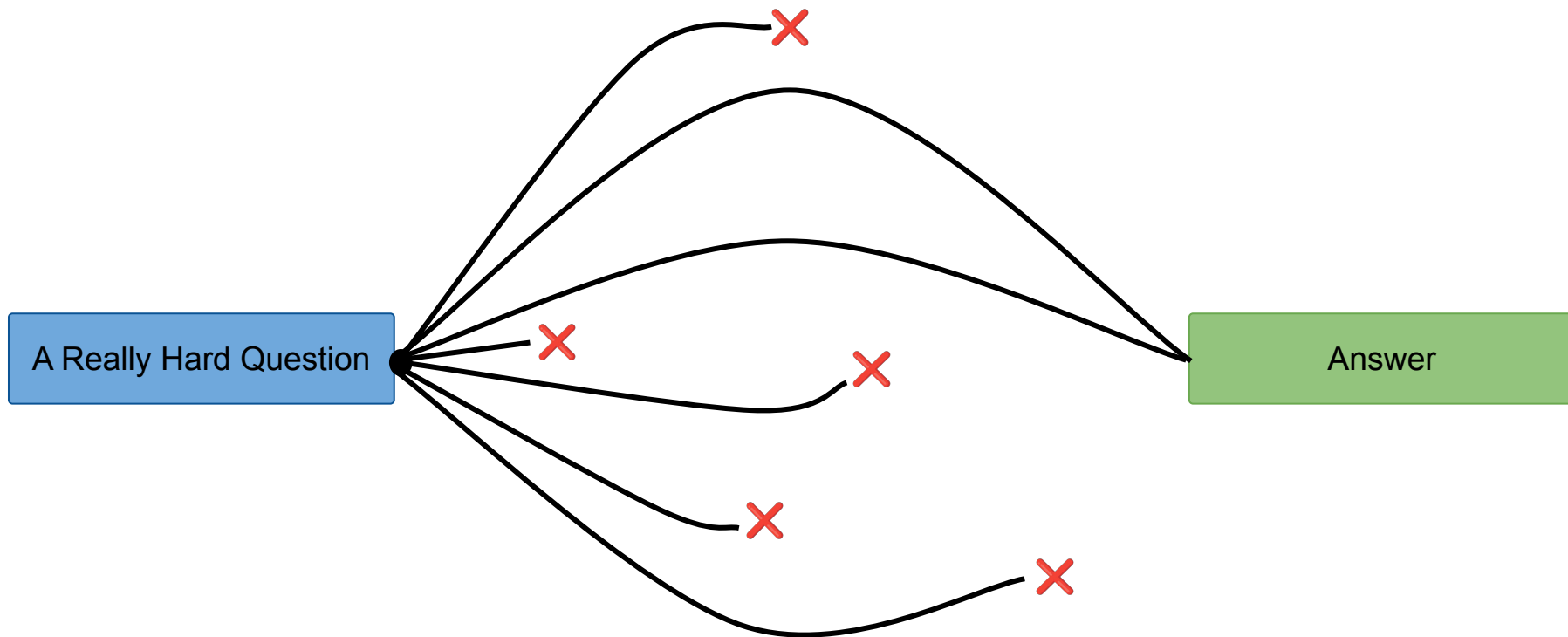
Reality: The reasoning path is not a straightforward path



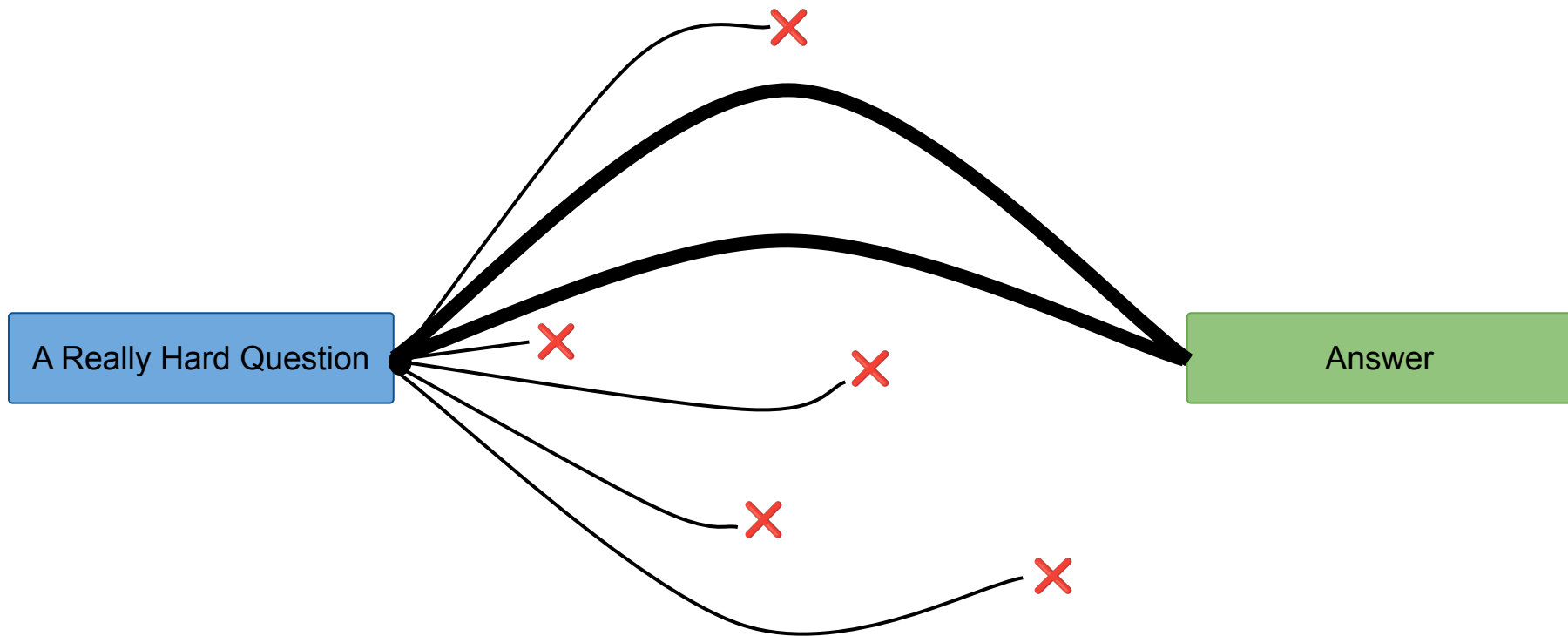
Finding the answer among all possible paths



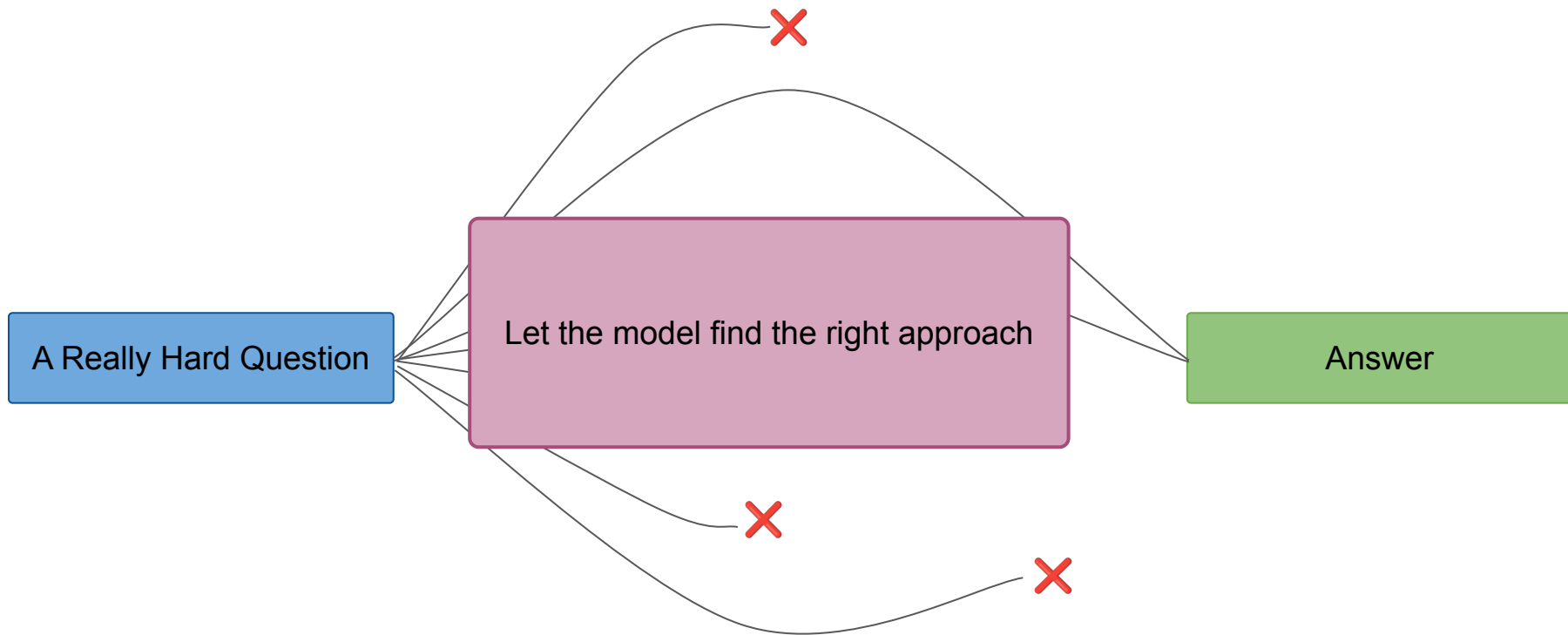
Don't hard code any rules in the model, let them learn!



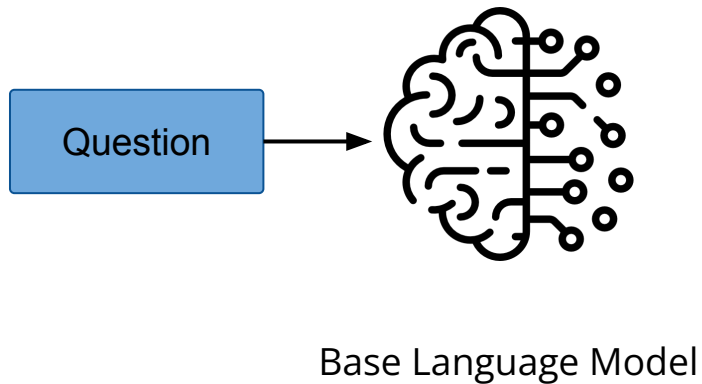
Don't hard code any rules in the model, let them learn!



Let the model reach the final answer, only verify the answer and give reward

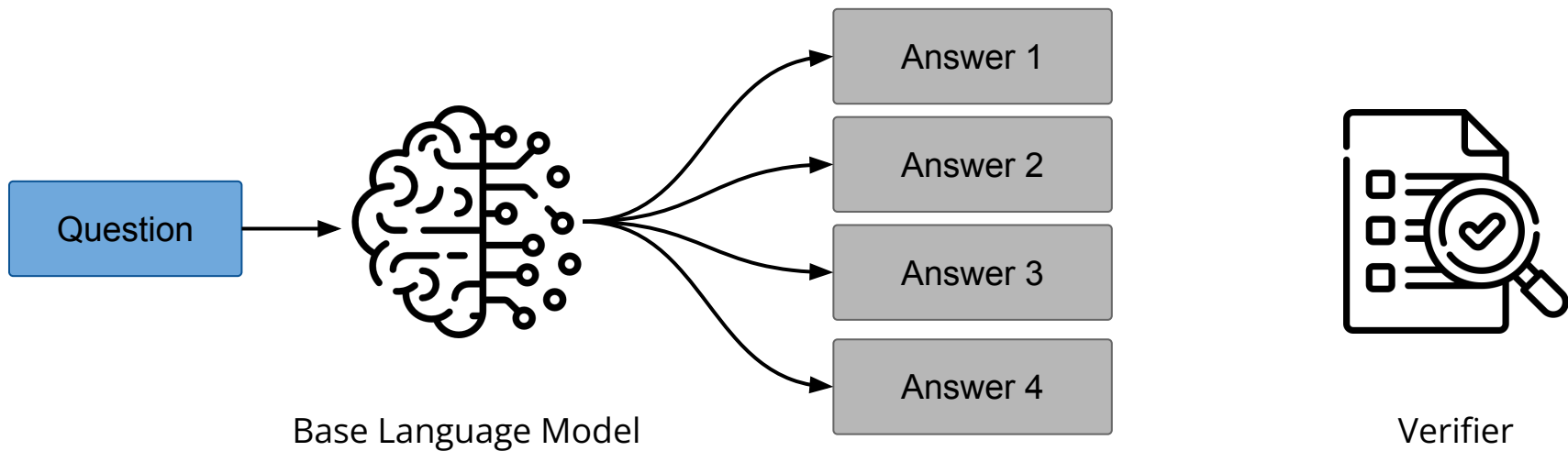


Making a Reasoning Model with Reinforcement Learning

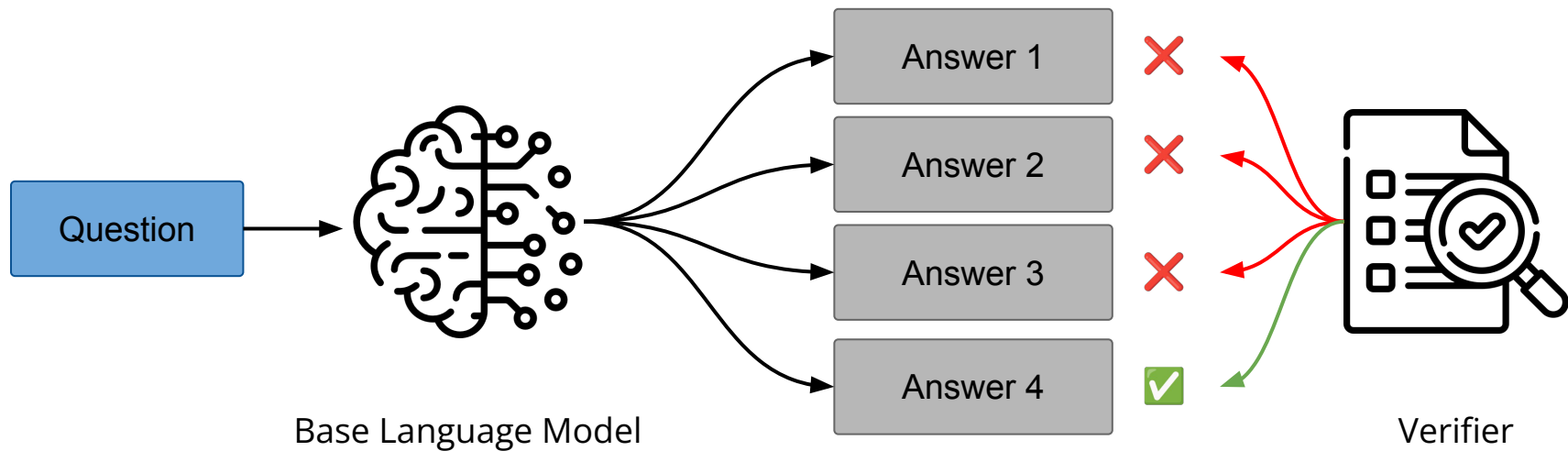


Verifier

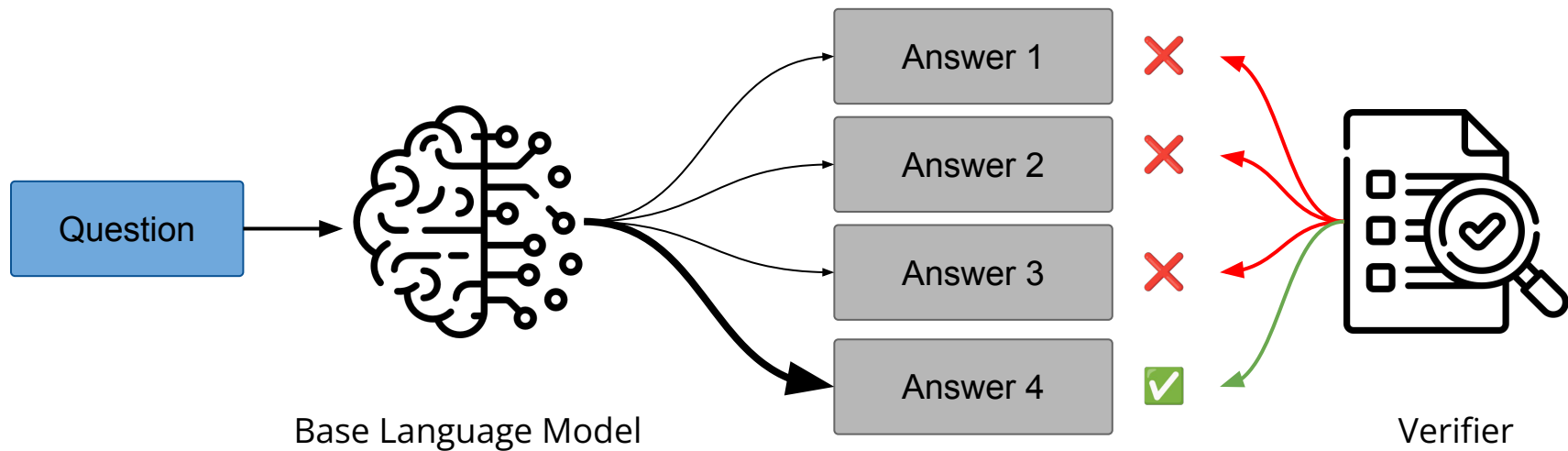
Making a Reasoning Model with Reinforcement Learning



Making a Reasoning Model with Reinforcement Learning



Making a Reasoning Model with Reinforcement Learning



Prompting tips for Reasoning Models

When to use reasoning models

Best for tasks requiring 5+ steps:

- Complex math, science, engineering problems
- Multi-step planning and strategy
- Decisions based on ambiguous information
- Tasks that would otherwise require human experts

Not ideal for:

- Simple factual questions (use GPT-4.1 instead)
- Real-time applications (thinking takes time)
- Tasks where latency matters more than accuracy

Tool calling with o3/o4-mini

Key improvements in o3/o4-mini (April 2025):

- Tools are used natively within chain-of-thought
- Supports up to ~100 tools with ~20 args each
- Reasoning items preserved across requests

Best practices:

- Don't ask the model to "plan more" before tool calls
- Use developer messages instead of system messages
- Let the model decide when/how to use tools autonomously
- In RAG: include only the most relevant context

Reasoning models can figure it out by themselves; helping them might distract them

- Keep prompts simple and direct
- Avoid chain-of-thought prompts
 - *“Let’s think step by step”* ✗
- Use delimiters for clarity
 - Use ### and < > tags
- Unnecessary Few-Shot Examples ✗

Agentic LLMs

From chatbots to autonomous
agents

What makes an LLM 'agentic'?

Not just generating text → **driving outcomes**

Key capabilities:

- Tool use (code execution, web browsing, file I/O, APIs)
- Multi-step planning and reasoning
- Self-correction and reflection
- Memory across interactions

An agentic LLM doesn't just respond—it ***acts***

From chatbot to agent

Chatbot (2022)

- Single turn
- Text in → Text out
- No memory between chats
- Cannot take actions

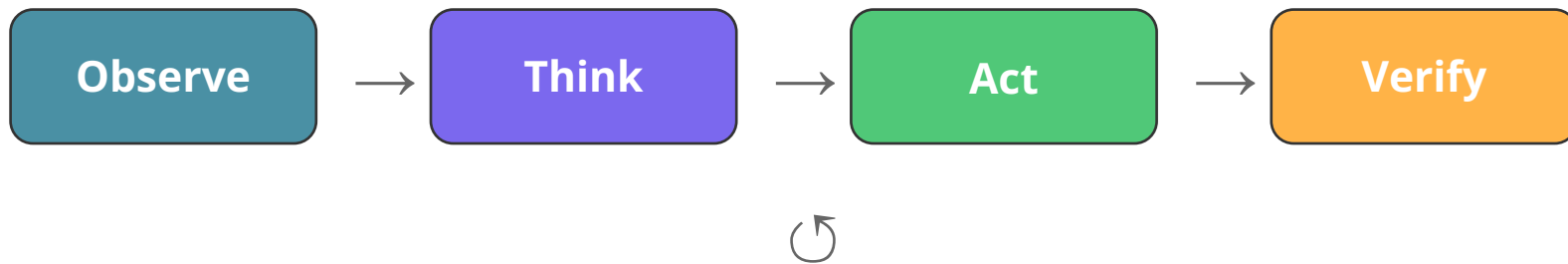


Agent (2025)

- Multi-turn workflows
- Execute code, browse web
- Read/write files, call APIs
- Plan → Act → Verify → Iterate

Examples: Claude Computer Use, OpenAI Operator, Google Mariner

The agent loop



ReAct pattern (Reasoning + Acting): The model explains its reasoning, then takes action, then observes results

This loop continues until the task is complete or the agent decides to ask for help

Coding agents: The killer app for agentic AI

Current state (Jan 2026):

- Claude Code achieves 72%+ on SWE-Bench (real GitHub issues)
- Can complete software tasks that take humans ~5 hours (METR)
- Work best with existing test suites for verification

Popular tools:

- Claude Code (Anthropic) – CLI tool, MCP integrations
- Cursor – IDE with Claude Sonnet, RAG-like context
- GitHub Copilot Workspace – integrated into GitHub
- Devin (Cognition) – autonomous software engineer

Computer use: Agents controlling screens

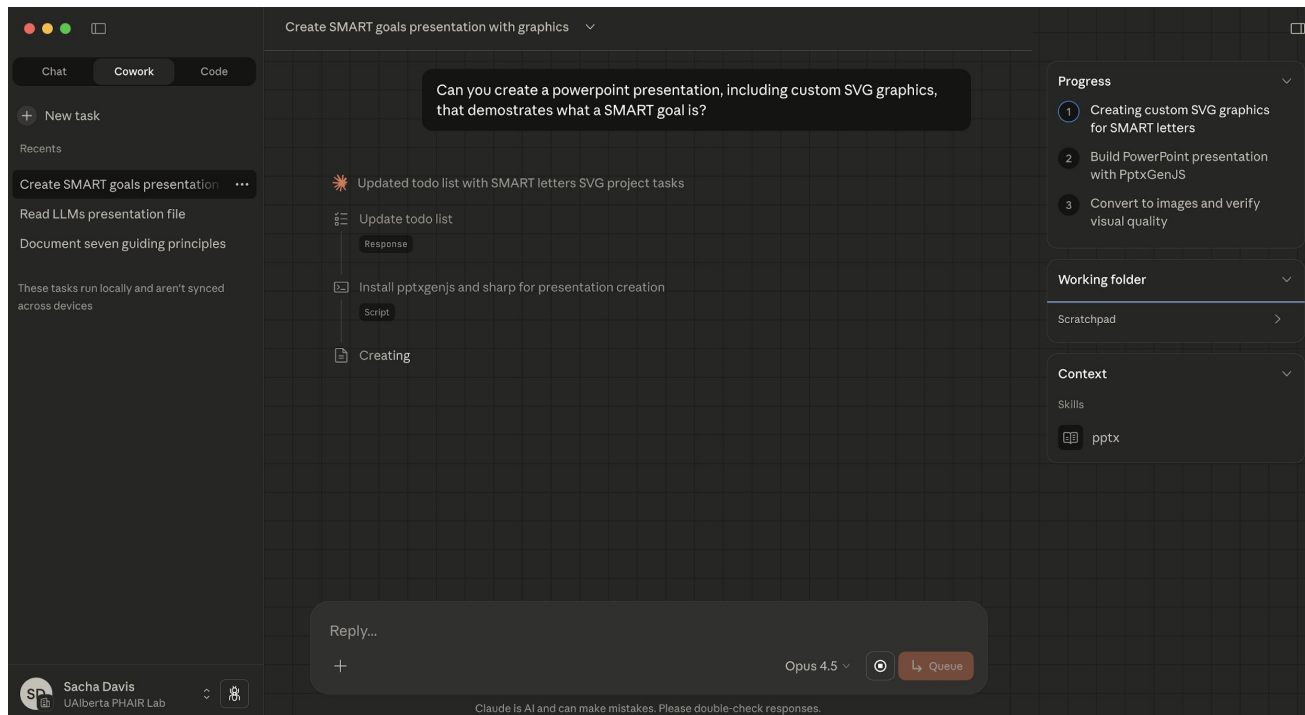
Model	WebArena	OSWorld
OpenAI Operator (CUA)	87%	38.1%
Google Mariner	83.5%	—
Claude Computer Use	56%	22%
Human baseline	—	72.4%

Key insight: These systems are impressive but still make mistakes, especially with unfamiliar interfaces

- Claude: broader (browser + desktop apps)
- Operator: simpler setup, browser-only

Agents in Action

Claude Cowork Demo



Prompt:
Can you create a PowerPoint presentation, including custom SVG graphics, that demonstrates what a SMART goal is?"

Getting started with agents (free tools)

Chat + Tools

ChatGPT Plus (\$20/mo)

- Web browsing, code exec

Claude Pro (\$20/mo)

- Computer use, artifacts

Gemini Advanced (\$20/mo)

- Deep Research mode

Free tiers available!

Coding Agents

Claude Code (free tier)

- CLI, works locally

Cursor (free tier)

- AI-native IDE

GitHub Copilot (free for students)

Windsurf (free tier)

Cody by Sourcegraph (free)

Build Your Own

Frameworks (all free/OSS):

- LangChain / LangGraph
- CrewAI
- OpenAI Agents SDK
- AutoGen (Microsoft)

No-code:

- n8n, Flowise, AgentGPT

Tip: Start with chat interfaces, then try coding agents, then build your own!

Limitations and risks

Current limitations:

- Compounding errors over long task horizons
- Slow execution (thinking + acting takes time)
- Struggle with unfamiliar interfaces
- Integration with existing systems (46% cite as primary challenge)

Safety concerns:

- Agents can take irreversible actions (delete files, send emails)
- Prompt injection: malicious instructions hidden in web pages/emails
- Accountability: Who is responsible when an agent makes a mistake?

Final Thoughts

Conclusion

- Language models are powerful AI systems capable of generating documents and conversations. However, they may produce errors or *hallucinations*.
- Reasoning models, such as o3 and R1, are powerful methods for solving difficult problems. However, their prompting techniques differ significantly from traditional language models.
- Agentic AI is the current promising direction for LLMs, having improved rapidly over the past year. However, there is room for growth and safety standards are still emerging.

Thank you!